

GestorPro

Codigo completo y logica del proyecto

Arquitectura MVC · Credenciales · Base de datos · Codigo fuente · Flujos clave

Sistema web de gestion de proyectos construido con **Flask**, **SQLAlchemy** y **PostgreSQL**. Este documento explica la arquitectura, muestra el **codigo fuente completo** de cada archivo del backend y el frontend con su logica comentada, detalla la base de datos y sus relaciones, e incluye las credenciales de las cuentas de prueba.

Autora	Anggie Alcedo
Repositorio	github.com/alcedoanggi-crypto/gestorpro
Generado	13/09/2026
Stack	Python · Flask 3 · SQLAlchemy 2 · PostgreSQL · Bootstrap 5 · Chart.js · ReportLab

Indice

- 1. Resumen ejecutivo — que es el proyecto y sus numeros.
- 2. **Credenciales de acceso** — usuarios y contraseñas para entrar a probarlo.
- 3. Arquitectura general (patron MVC) y que hace create_app().
- 4. Mapa de modulos — que ruta / funcion hace que (tablas resumen).
- 5. Base de datos — diagrama entidad-relacion, tablas, relaciones y reglas de negocio.
- 6. **Codigo fuente completo** — cada archivo de app/, leído tal cual esta en el repositorio, con una explicacion de su logica.
- 7. Flujos clave paso a paso (login, modales AJAX, Gantt, reportes, notificaciones).
- 8. Preguntas frecuentes para explicarlo o defenderlo.

1 · Resumen ejecutivo

GestorPro es una aplicacion web tipo *SaaS interno* para gestionar proyectos: crear proyectos y tareas, asignarlas a personas y equipos, seguir el avance en un cronograma de Gantt, medir el cumplimiento de plazos y exportar reportes. Esta pensada para tres tipos de usuario -administrador, gerente y colaborador- cada uno con una vista y unos permisos distintos sobre los mismos datos.

El objetivo tecnico del proyecto fue construir una aplicacion **completa de extremo a extremo** -modelo de datos, autenticacion, autorizacion por objeto, interfaz, reportes y exportacion- usando un stack clasico de servidor (Flask + Jinja2) en vez de un framework de frontend, y demostrando patrones de arquitectura limpios: separacion en capas, permisos centralizados, y una capa de servicios reutilizable fuera de las rutas.

9 modelos de datos	8 blueprints (modulos de rutas)	4 servicios de negocio
~45 rutas HTTP	10 pruebas automatizadas	3 roles de acceso

2 · Credenciales de acceso (datos de prueba)

Estas cuentas **no son secretas de produccion**: las crea el comando `flask seed` (archivo `app/seeds.py`) solo para poder probar la aplicacion con datos realistas. Las siete comparten la misma contraseña de prueba para que sea facil recordarlas al hacer una demo.

Nombre	Correo	Contraseña	Rol	Cargo
Ana Torres	admin@demo.com	Demo1234	admin	Directora de PMO
Bruno Diaz	gerente@demo.com	Demo1234	gerente	Gerente de Proyectos
Carla Ruiz	carla@demo.com	Demo1234	gerente	Gerente de Producto
Diego Lopez	diego@demo.com	Demo1234	colaborador	Desarrollador Backend
Elena Marin	elena@demo.com	Demo1234	colaborador	Desarrolladora Frontend
Franco Vega	franco@demo.com	Demo1234	colaborador	Disenador UX
Gabriela Sol	gabriela@demo.com	Demo1234	colaborador	QA Engineer

Como se crean y se verifican (sin guardar la contraseña en claro)

- `seeds.py` llama `user.set_password("Demo1234")` por cada cuenta. Esa funcion (en `models/user.py`) usa `generate_password_hash` de Werkzeug con el algoritmo **pbkdf2:sha256**: la contraseña en texto plano nunca llega a la base de datos, solo se guarda su hash en la columna `password_hash`.
- Al iniciar sesion (`controllers/auth.py`), `user.check_password(...)` vuelve a aplicar el mismo hash a lo que escribio la persona y compara los resultados -nunca se "desencripta" nada, un hash no se puede revertir.
- El **primer usuario que se registra** en una instalacion nueva se vuelve `admin` automaticamente; todos los siguientes entran como `colaborador` (se cambia el rol despues desde Usuarios).
- Una contraseña nueva (registro o "olvide mi contraseña") debe tener minimo 8 caracteres, al menos una letra y al menos un numero (validadores en `forms/auth_forms.py`).
- "Olvide mi contraseña" genera un token aleatorio de un solo uso (`secrets.token_urlsafe(32)`) valido 1 hora (`User.generate_reset_token`); el enlace se envia por correo y, si SMTP no esta configurado, se imprime en la consola del servidor para poder probarlo en local.

Importante: estas credenciales solo existen si corriste `flask seed` en tu base de datos local/de pruebas. Antes de publicar la app en internet: cambia `SECRET_KEY` en `.env`, no subas ese archivo al repositorio (ya esta en `.gitignore`), y no dejes estas cuentas de prueba en una base de datos real.

3 · Arquitectura general (patron MVC)

El código separa responsabilidades en tres capas clásicas, más dos capas propias (**servicios** y **utils**) para que las rutas no acumulen lógica de negocio:

Capa	Carpeta	Responsabilidad
Modelo (M)	app/models/	Tablas SQLAlchemy, relaciones entre ellas y propiedades calculadas.
Vista (V)	app/templates/ + app/static/	Plantillas Jinja2, hojas de estilo y JavaScript. No contienen lógica de negocio.
Controlador (C)	app/controllers/	Blueprints de Flask: reciben la petición, comprueban permisos, llaman a servicios/modelos y deciden que devolver.
Servicios	app/services/	Lógica de negocio reutilizable que no depende de una request concreta.
Utilidades	app/utils/	Permisos por objeto, filtros de consulta por rol, decoradores y el patrón de modales.

```
app/  
|-- __init__.py      application factory  
|-- config.py        configuracion por entorno (dev / test / prod)  
|-- extensions.py    instancias unicas: db, login_manager, csrf, mail, migrate  
|-- seeds.py         datos de demostracion (flask seed)  
|-- models/          capa M -> SQLAlchemy  
|-- controllers/     capa C -> blueprints (auth, dashboard, projects, tasks,  
|                   teams, users, reports, api)  
|-- services/        logica de negocio: report_service, export_service,  
|                   notification_service, mail_service  
|-- forms/           validacion con Flask-WTF  
|-- utils/           permisos, scoping por rol, decoradores, helpers de modal  
|-- templates/       capa V -> Jinja2 (base + una carpeta por modulo)  
|-- static/          css/style.css + js/ (app, dashboard, gantt, reports, modal-form)
```

4 · Mapa de modulos (resumen antes del codigo)

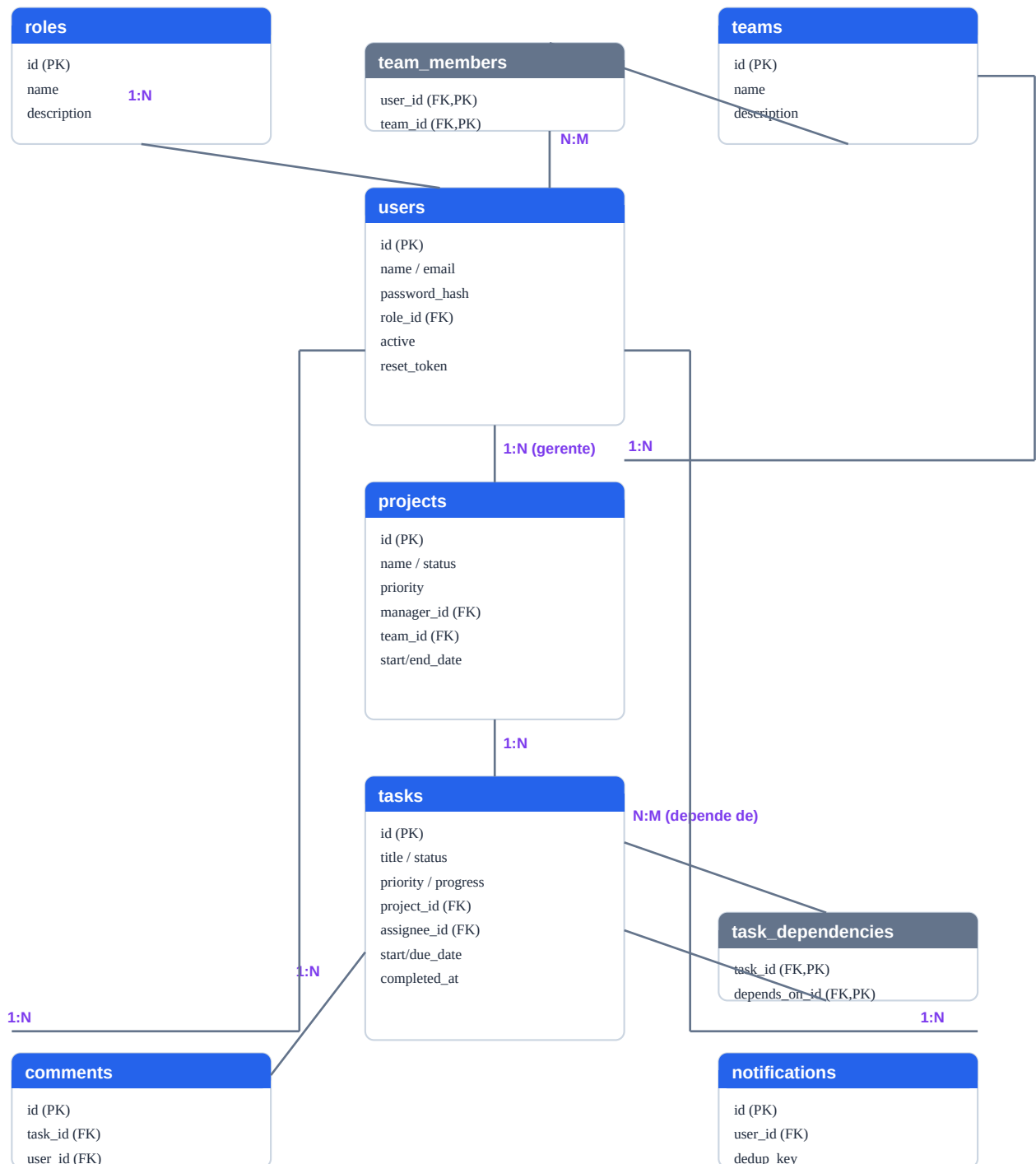
Antes de leer el codigo completo (seccion 6), esta tabla ubica que hace cada blueprint para poder saltar directo a lo que interese:

Blueprint	Prefijo	Responsabilidad
auth	/ (login, registro...)	Inicio de sesion, alta del primer usuario, recuperacion de contrasena.
dashboard	/	Pantalla de inicio: KPIs, graficos, mis tareas, proyectos recientes.
projects	/proyectos	CRUD de proyectos, vista de Gantt, exportacion a PDF.
tasks	/tareas	CRUD de tareas, comentarios, cambio de estado, dependencias.
teams	/equipos	CRUD de equipos e integrantes.
users	/usuarios	CRUD de usuarios, activar/desactivar, perfil propio.
reports	/reportes	Graficos agregados y exportacion a Excel/PDF.
api	/api	Endpoints JSON: Gantt drag&drop,, progreso, notificaciones.

5 · Base de datos

El modelo de datos vive en `app/models/` con SQLAlchemy (un ORM: cada clase Python es una tabla). Hay **7 tablas propias** y **2 tablas puente** para relaciones muchos-a-muchos. El código real de cada modelo está en la sección 6.2.

5.1 Diagrama entidad-relacion



Lectura del diagrama: las cajas grises (`team_members` y `task_dependencies`) son tablas puente sin modelo propio -se declaran como `db.Table(...)`, no como clases- porque solo guardan las dos claves foraneas de la relacion.

5.2 Relaciones — 1:N y N:M

Uno-a-muchos (1:N): se resuelve con una clave foranea simple, por ejemplo `Project.manager_id -> User.id` (un usuario/gerente puede tener muchos proyectos gestionados). **Muchos-a-muchos (N:M):** se resuelve con una tabla puente pasada como `secondary` en la relacion; hay dos casos: usuarios<->equipos (`team_members`) y tareas<->tareas auto-referencial para las dependencias del Gantt (`task_dependencies`). El codigo exacto esta en la seccion 6.2 (`user.py` y `task.py`).

5.3 Datos que NO se guardan: propiedades calculadas

Propiedad	Se calcula asi
<code>Project.progress</code>	tareas completadas / total de tareas del proyecto, redondeado a %
<code>Project.is_delayed</code>	<code>end_date</code> ya paso y el proyecto no esta completado ni cancelado
<code>Task.is_overdue</code>	<code>due_date</code> ya paso y la tarea no esta 'completada'
<code>Task.to_gantt()</code>	arma el diccionario que necesita Frappe Gantt: fechas, % de avance y dependencias
<code>User.initials</code>	iniciales del nombre para el avatar circular

6 · Código fuente completo, archivo por archivo

Esta sección reproduce **el código real** de cada archivo de app/ (leído directamente del repositorio al generar este documento) con una explicación de su lógica antes de cada bloque. Las plantillas HTML y las hojas de estilo no se listan completas -son presentación, no lógica- pero se explican en la sección 3.

6.1 Arranque de la aplicacion

wsgi.py (11 lineas)

Punto de entrada WSGI. Carga variables de entorno (.env) y crea la app con create_app(); es lo que ejecutan `python wsgi.py` y Unicorn en produccion.

```
"""Punto de entrada WSGI. Uso: flask run / gunicorn wsgi:app"""
from dotenv import load_dotenv

load_dotenv()

from app import create_app # noqa: E402

app = create_app()

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

app/__init__.py (143 lineas)

El application factory: ensambla extensiones, blueprints, filtros de Jinja, el contexto global (notificaciones), las paginas de error y los comandos de consola (init-db, seed, scan-notifications).

```
"""Application factory. Ensambla configuracion, extensiones, blueprints y CLI."""
import os
from datetime import datetime

from flask import Flask, render_template
from flask_login import current_user

from .config import config
from .extensions import csrf, db, login_manager, mail, migrate

def create_app(config_name=None):
    config_name = config_name or os.environ.get("FLASK_CONFIG", "default")
    app = Flask(__name__)
    app.config.from_object(config[config_name])

    _init_extensions(app)
    _register_blueprints(app)
    _register_jinja(app)
    _register_context(app)
    _register_errors(app)
    _register_cli(app)
    return app

def _init_extensions(app):
    db.init_app(app)
    migrate.init_app(app, db)
    login_manager.init_app(app)
    csrf.init_app(app)
    mail.init_app(app)

    from .models import User

    @login_manager.user_loader
    def load_user(user_id):
        return db.session.get(User, int(user_id))

def _register_blueprints(app):
    from .controllers.api import api_bp
    from .controllers.auth import auth_bp
    from .controllers.dashboard import dashboard_bp
    from .controllers.projects import projects_bp
    from .controllers.reports import reports_bp
    from .controllers.tasks import tasks_bp
    from .controllers.teams import teams_bp
    from .controllers.users import users_bp

    app.register_blueprint(auth_bp)
    app.register_blueprint(dashboard_bp)
    app.register_blueprint(projects_bp, url_prefix="/proyectos")
    app.register_blueprint(tasks_bp, url_prefix="/tareas")
    app.register_blueprint(users_bp, url_prefix="/usuarios")
    app.register_blueprint(teams_bp, url_prefix="/equipos")
```

```

app.register_blueprint(reports_bp, url_prefix="/reportes")
app.register_blueprint(api_bp, url_prefix="/api")
csrf.exempt(api_bp) # las llamadas JSON usan sesion + header propio

def _register_jinja(app):
    from .models.enums import Priority, ProjectStatus, TaskStatus

    colors = {
        ProjectStatus.ACTIVE: "success", ProjectStatus.PLANNING: "info",
        ProjectStatus.ON_HOLD: "warning", ProjectStatus.COMPLETED: "secondary",
        ProjectStatus.CANCELLED: "danger",
        TaskStatus.TODO: "secondary", TaskStatus.IN_PROGRESS: "primary",
        TaskStatus.REVIEW: "warning", TaskStatus.DONE: "success",
        Priority.LOW: "success", Priority.MEDIUM: "info",
        Priority.HIGH: "warning", Priority.CRITICAL: "danger",
    }

    @app.template_filter("fecha")
    def _fecha(value, fmt="%d/%m/%Y"):
        return value.strftime(fmt) if value else "-"

    @app.template_filter("desde")
    def _desde(value):
        if not value:
            return ""
        delta = datetime.utcnow() - value
        s = delta.total_seconds()
        if s < 60:
            return "hace instantes"
        if s < 3600:
            return f"hace {int(s // 60)} min"
        if s < 86400:
            return f"hace {int(s // 3600)} h"
        return f"hace {delta.days} d"

    app.jinja_env.globals.update(
        bcolor=lambda key: colors.get(key, "secondary"),
        ProjectStatus=ProjectStatus,
        TaskStatus=TaskStatus,
        Priority=Priority,
    )

def _register_context(app):
    from .services.notification_service import unread_count

    @app.context_processor
    def inject_globals():
        ctx = {"now": datetime.utcnow(), "nav_notifications": [], "nav_unread": 0}
        if current_user.is_authenticated:
            try:
                ctx["nav_notifications"] = current_user.notifications[:8]
                ctx["nav_unread"] = unread_count(current_user)
            except Exception: # p. ej. sesion de BD rota durante un error 500
                db.session.rollback()
        return ctx

def _register_errors(app):
    for code in (403, 404, 500):
        app.register_error_handler(
            code,
            lambda e, code=code: (render_template(f"errors/{code}.html"), code),
        )

def _register_cli(app):
    from .models import Role
    from .seeds import run_seed
    from .services.notification_service import scan_due_soon

    @app.cli.command("init-db")
    def init_db():
        """Crea las tablas y los roles base."""
        db.create_all()
        Role.seed_defaults()
        print("Base de datos lista.")

    @app.cli.command("seed")
    def seed():
        """Carga datos de demostracion."""
        run_seed()

```

```

@app.cli.command("scan-notifications")
def scan_notifications():
    """Genera notificaciones de tareas por vencer (usar en cron)."""
    print(f"{scan_due_soon()} notificaciones generadas.")

```

app/config.py (60 líneas)

Configuración por entorno (Development / Production / Testing) leída de variables de entorno, con valores por defecto seguros para desarrollo.

```

"""Configuración por entorno."""
import os
from datetime import timedelta

class Config:
    SECRET_KEY = os.environ.get("SECRET_KEY", "dev-secret-change-me")

    # Driver psycopg 3: prefijo "postgresql+psycopg://"
    SQLALCHEMY_DATABASE_URI = os.environ.get(
        "DATABASE_URL",
        "postgresql+psycopg://postgres:postgres@localhost:5432/gestorpro",
    )
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    SQLALCHEMY_ENGINE_OPTIONS = {"pool_pre_ping": True}

    PERMANENT_SESSION_LIFETIME = timedelta(days=7)
    SESSION_COOKIE_HTTPONLY = True
    SESSION_COOKIE_SAMESITE = "Lax"
    WTF_CSRF_TIME_LIMIT = None

    # Correo
    MAIL_SERVER = os.environ.get("MAIL_SERVER", "localhost")
    MAIL_PORT = int(os.environ.get("MAIL_PORT", 25))
    MAIL_USE_TLS = os.environ.get("MAIL_USE_TLS", "false").lower() == "true"
    MAIL_USERNAME = os.environ.get("MAIL_USERNAME") or None
    MAIL_PASSWORD = os.environ.get("MAIL_PASSWORD") or None
    MAIL_DEFAULT_SENDER = os.environ.get(
        "MAIL_DEFAULT_SENDER", "GestorPro <no-reply@gestorpro.local>"
    )

    # Negocio
    TASK_DUE_SOON_DAYS = int(os.environ.get("TASK_DUE_SOON_DAYS", 3))
    ITEMS_PER_PAGE = 12

class DevelopmentConfig(Config):
    DEBUG = True

class ProductionConfig(Config):
    DEBUG = False
    SESSION_COOKIE_SECURE = True

class TestingConfig(Config):
    TESTING = True
    SQLALCHEMY_DATABASE_URI = os.environ.get(
        "TEST_DATABASE_URL", "sqlite:///memory:"
    )
    WTF_CSRF_ENABLED = False
    MAIL_SUPPRESS_SEND = True

config = {
    "development": DevelopmentConfig,
    "production": ProductionConfig,
    "testing": TestingConfig,
    "default": DevelopmentConfig,
}

```

app/extensions.py (16 lineas)

Instancias unicas de SQLAlchemy, Flask-Migrate, Flask-Login, CSRF y Flask-Mail, sin atarlas todavia a ninguna app (se inicializan despues en create_app, patron 'application factory').

```
"""Instancias de extensiones Flask (patron application factory)."""
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate
from flask_login import LoginManager
from flask_wtf import CSRFProtect
from flask_mail import Mail

db = SQLAlchemy()
migrate = Migrate()
login_manager = LoginManager()
csrf = CSRFProtect()
mail = Mail()

login_manager.login_view = "auth.login"
login_manager.login_message = "Inicia sesion para continuar."
login_manager.login_message_category = "warning"
```

app/seeds.py (117 lineas)

Genera los datos de demostracion: los 7 usuarios de la seccion 2, 2 equipos, 3 proyectos y 10 tareas con dependencias y comentarios de ejemplo.

```
"""Datos de demostracion. Uso: flask seed"""
from datetime import date, datetime, timedelta

from .extensions import db
from .models import (Comment, Project, Role, Task, Team, User)
from .models.enums import Priority, ProjectStatus, TaskStatus

def run_seed():
    db.create_all()
    Role.seed_defaults()

    if User.query.count() > 0:
        print("Ya hay datos. Aborta el seed para no duplicar.")
        return

    admin_r = Role.get(Role.ADMIN)
    manager_r = Role.get(Role.MANAGER)
    collab_r = Role.get(Role.COLLABORATOR)

    def mkuser(name, email, role, title):
        u = User(name=name, email=email, role=role, job_title=title)
        u.set_password("Demo1234")
        db.session.add(u)
        return u

    admin = mkuser("Ana Torres", "admin@demo.com", admin_r, "Directora de PMO")
    gerente1 = mkuser("Bruno Diaz", "gerente@demo.com", manager_r, "Gerente de Proyectos")
    gerente2 = mkuser("Carla Ruiz", "carla@demo.com", manager_r, "Gerente de Producto")
    dev1 = mkuser("Diego Lopez", "diego@demo.com", collab_r, "Desarrollador Backend")
    dev2 = mkuser("Elena Marin", "elena@demo.com", collab_r, "Desarrolladora Frontend")
    des1 = mkuser("Franco Vega", "franco@demo.com", collab_r, "Disenador UX")
    qa1 = mkuser("Gabriela Sol", "gabriela@demo.com", collab_r, "QA Engineer")
    db.session.flush()

    t_web = Team(name="Plataforma Web", description="Equipo full-stack del producto principal")
    t_web.members = [gerente1, dev1, dev2, des1, qa1]
    t_mobile = Team(name="Movil", description="Aplicaciones iOS y Android")
    t_mobile.members = [gerente2, dev2, qa1]
    db.session.add_all([t_web, t_mobile])
    db.session.flush()

    today = date.today()

    p1 = Project(
        name="Rediseño del portal de clientes",
        description="Nueva experiencia de usuario y migración a componentes reutilizables.",
        status=ProjectStatus.ACTIVE, priority=Priority.HIGH,
        start_date=today - timedelta(days=20), end_date=today + timedelta(days=40),
        manager=gerente1, team=t_web,
    )
    p2 = Project(
```

```

        name="App movil v2",
        description="Rework de la app con modo offline y notificaciones push.",
        status=ProjectStatus.PLANNING, priority=Priority.CRITICAL,
        start_date=today + timedelta(days=5), end_date=today + timedelta(days=90),
        manager=gerente2, team=t_mobile,
    )
    p3 = Project(
        name="Integracion de facturacion",
        description="Conectar con el ERP y automatizar la emision de facturas.",
        status=ProjectStatus.ON_HOLD, priority=Priority.MEDIUM,
        start_date=today - timedelta(days=60), end_date=today - timedelta(days=5),
        manager=gerente1, team=t_web,
    )
    db.session.add_all([p1, p2, p3])
    db.session.flush()

def mktask(project, title, assignee, status, prio, d_start, d_due, progress=0, deps=None):
    t = Task(
        title=title, project=project, assignee=assignee, status=status, priority=prio,
        start_date=d_start, due_date=d_due, progress=progress, created_by_id=admin.id,
        estimated_hours=16,
    )
    if status == TaskStatus.DONE:
        t.completed_at = datetime.combine(d_due, datetime.min.time())
        t.progress = 100
    if deps:
        t.dependencies = deps
    db.session.add(t)
    return t

a1 = mktask(p1, "Auditoria UX del portal actual", des1, TaskStatus.DONE, Priority.MEDIUM,
            today - timedelta(days=20), today - timedelta(days=12), 100)
a2 = mktask(p1, "Wireframes de alta fidelidad", des1, TaskStatus.DONE, Priority.HIGH,
            today - timedelta(days=12), today - timedelta(days=4), 100, deps=[a1])
a3 = mktask(p1, "Sistema de componentes en React", dev2, TaskStatus.IN_PROGRESS, Priority.HIGH,
            today - timedelta(days=4), today + timedelta(days=10), 55, deps=[a2])
a4 = mktask(p1, "API de perfil de cliente", dev1, TaskStatus.IN_PROGRESS, Priority.HIGH,
            today - timedelta(days=2), today + timedelta(days=8), 30)
a5 = mktask(p1, "Pruebas end-to-end", qa1, TaskStatus.TODO, Priority.MEDIUM,
            today + timedelta(days=10), today + timedelta(days=25), 0, deps=[a3, a4])
a6 = mktask(p1, "Corregir bug de login (vencida)", dev1, TaskStatus.IN_PROGRESS, Priority.CRITICAL,
            today - timedelta(days=6), today - timedelta(days=1), 40)

b1 = mktask(p2, "Definir arquitectura offline-first", gerente2, TaskStatus.TODO, Priority.CRITICAL,
            today + timedelta(days=5), today + timedelta(days=15), 0)
b2 = mktask(p2, "Prototipo de sincronizacion", dev2, TaskStatus.TODO, Priority.HIGH,
            today + timedelta(days=15), today + timedelta(days=35), 0, deps=[b1])

c1 = mktask(p3, "Mapa de campos ERP <-> sistema", dev1, TaskStatus.DONE, Priority.MEDIUM,
            today - timedelta(days=60), today - timedelta(days=45), 100)
c2 = mktask(p3, "Cliente HTTP del ERP", dev1, TaskStatus.REVIEW, Priority.MEDIUM,
            today - timedelta(days=44), today - timedelta(days=10), 80, deps=[c1])

db.session.flush()
db.session.add_all([
    Comment(task=a3, author=dev2, body="Avanzando con los formularios, faltan estados de error."),
    Comment(task=a3, author=gerente1, body="Prioriza el datepicker, lo necesita el equipo de API."),
    Comment(task=a6, author=dev1, body="El bug depende de la nueva libreria de auth, investigando."),
])

db.session.commit()
print("Seed completo.")
print(" Admin:      admin@demo.com    / Demo1234")
print(" Gerente:    gerente@demo.com   / Demo1234")
print(" Colab.:      diego@demo.com     / Demo1234")

```

6.2 Modelos (capa M)

app/models/__init__.py (24 lineas)

Reexporta todos los modelos para poder hacer `from app.models import Task, Project...` en el resto del código.

```
"""Capa de modelos (M del patron MVC)."""
from .enums import ProjectStatus, TaskStatus, Priority
from .role import Role
from .user import User, team_members
from .team import Team
from .project import Project
from .task import Task, task_dependencies
from .comment import Comment
from .notification import Notification

__all__ = [
    "ProjectStatus",
    "TaskStatus",
    "Priority",
    "Role",
    "User",
    "team_members",
    "Team",
    "Project",
    "Task",
    "task_dependencies",
    "Comment",
    "Notification",
]
```

app/models/enums.py (57 lineas)

Estados de dominio (ProjectStatus, TaskStatus, Priority) como clases con constantes de texto -no un ENUM nativo de PostgreSQL- para poder añadir un estado sin migrar el tipo de columna.

```
"""Enumeraciones de dominio compartidas por los modelos."""

class ProjectStatus:
    PLANNING = "planificacion"
    ACTIVE = "activo"
    ON_HOLD = "en_pausa"
    COMPLETED = "completado"
    CANCELLED = "cancelado"

    ALL = [PLANNING, ACTIVE, ON_HOLD, COMPLETED, CANCELLED]
    LABELS = {
        PLANNING: "Planificacion",
        ACTIVE: "Activo",
        ON_HOLD: "En pausa",
        COMPLETED: "Completado",
        CANCELLED: "Cancelado",
    }
    CLOSED = {COMPLETED, CANCELLED}

    @classmethod
    def choices(cls):
        return [(v, cls.LABELS[v]) for v in cls.ALL]

class TaskStatus:
    TODO = "pendiente"
    IN_PROGRESS = "en_progreso"
    REVIEW = "en_revision"
    DONE = "completada"

    ALL = [TODO, IN_PROGRESS, REVIEW, DONE]
    LABELS = {
        TODO: "Pendiente",
        IN_PROGRESS: "En progreso",
        REVIEW: "En revision",
        DONE: "Completada",
    }

    @classmethod
    def choices(cls):
        return [(v, cls.LABELS[v]) for v in cls.ALL]

class Priority:
```

```

LOW = "baja"
MEDIUM = "media"
HIGH = "alta"
CRITICAL = "critica"

ALL = [LOW, MEDIUM, HIGH, CRITICAL]
LABELS = {LOW: "Baja", MEDIUM: "Media", HIGH: "Alta", CRITICAL: "Critica"}
WEIGHT = {LOW: 1, MEDIUM: 2, HIGH: 3, CRITICAL: 4}

@classmethod
def choices(cls):
    return [(v, cls.LABELS[v]) for v in cls.ALL]

```

app/models/role.py (35 lineas)

Catalogo de 3 roles fijos y el metodo que los siembra la primera vez.

```

from ..extensions import db

class Role(db.Model):
    __tablename__ = "roles"

    ADMIN = "admin"
    MANAGER = "gerente"
    COLLABORATOR = "colaborador"

    DEFAULTS = {
        ADMIN: "Acceso total: usuarios, configuracion y todos los proyectos.",
        MANAGER: "Gestiona sus proyectos, equipos, tareas y reportes.",
        COLLABORATOR: "Trabaja sobre las tareas que tiene asignadas.",
    }

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(50), unique=True, nullable=False)
    description = db.Column(db.String(255))

    users = db.relationship("User", back_populates="role")

    @staticmethod
    def seed_defaults():
        for name, desc in Role.DEFAULTS.items():
            if not Role.query.filter_by(name=name).first():
                db.session.add(Role(name=name, description=desc))
            db.session.commit()

    @staticmethod
    def get(name):
        return Role.query.filter_by(name=name).first()

    def __repr__(self):
        return f"<Role {self.name}>"

```

app/models/user.py (105 lineas)

El modelo de cuenta: hash de contrasena, relaciones con equipos, proyectos gestionados, tareas asignadas y notificaciones, y el flujo completo de recuperacion de contrasena por token.

```

import secrets
from datetime import datetime, timedelta

from flask_login import UserMixin
from werkzeug.security import generate_password_hash, check_password_hash

from ..extensions import db
from .role import Role

# Relacion N:M usuarios <-> equipos
team_members = db.Table(
    "team_members",
    db.Column("user_id", db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"), primary_key=True),
    db.Column("team_id", db.Integer, db.ForeignKey("teams.id", ondelete="CASCADE"), primary_key=True),
)

class User(UserMixin, db.Model):
    __tablename__ = "users"

    id = db.Column(db.Integer, primary_key=True)

```

```

name = db.Column(db.String(120), nullable=False)
email = db.Column(db.String(120), unique=True, nullable=False, index=True)
password_hash = db.Column(db.String(255), nullable=False)
role_id = db.Column(db.Integer, db.ForeignKey("roles.id"), nullable=False)
job_title = db.Column(db.String(120))
avatar_url = db.Column(db.String(255))
active = db.Column("is_active", db.Boolean, default=True, nullable=False)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)
last_login_at = db.Column(db.DateTime)

reset_token = db.Column(db.String(255), index=True)
reset_token_expiry = db.Column(db.DateTime)

role = db.relationship("Role", back_populates="users")
teams = db.relationship("Team", secondary=team_members, back_populates="members")
managed_projects = db.relationship(
    "Project", back_populates="manager", foreign_keys="Project.manager_id"
)
assigned_tasks = db.relationship(
    "Task", back_populates="assignee", foreign_keys="Task.assignee_id"
)
comments = db.relationship("Comment", back_populates="author", cascade="all, delete-orphan")
notifications = db.relationship(
    "Notification",
    back_populates="user",
    cascade="all, delete-orphan",
    order_by="Notification.created_at.desc()",
)

# --- Contraseña ---
def set_password(self, raw):
    self.password_hash = generate_password_hash(raw)

def check_password(self, raw):
    return check_password_hash(self.password_hash, raw)

# --- Roles ---
@property
def role_name(self):
    return self.role.name if self.role else None

def has_role(self, *names):
    return self.role_name in names

@property
def is_admin(self):
    return self.role_name == Role.ADMIN

@property
def is_manager(self):
    return self.role_name == Role.MANAGER

# Flask-Login usa esta propiedad para permitir el acceso
@property
def is_active(self):
    return bool(self.active)

# --- Recuperación de contraseña ---
def generate_reset_token(self, ttl_hours=1):
    self.reset_token = secrets.token_urlsafe(32)
    self.reset_token_expiry = datetime.utcnow() + timedelta(hours=ttl_hours)
    return self.reset_token

def verify_reset_token(self, token):
    return bool(
        self.reset_token
        and token
        and secrets.compare_digest(self.reset_token, token)
        and self.reset_token_expiry
        and self.reset_token_expiry > datetime.utcnow()
    )

def clear_reset_token(self):
    self.reset_token = None
    self.reset_token_expiry = None

# --- Presentación ---
@property
def initials(self):
    parts = [p for p in self.name.split() if p]
    return ("".join(p[0] for p in parts[:2]) or "?").upper()

def __repr__(self):

```

```
return f"<User {self.email}>"
```

app/models/team.py (23 lineas)

Equipos de trabajo; relacion N:M con usuarios via team_members.

```
from datetime import datetime

from ..extensions import db
from .user import team_members

class Team(db.Model):
    __tablename__ = "teams"

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(120), unique=True, nullable=False)
    description = db.Column(db.Text)
    created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

    members = db.relationship("User", secondary=team_members, back_populates="teams")
    projects = db.relationship("Project", back_populates="team")

    @property
    def member_count(self):
        return len(self.members)

    def __repr__(self):
        return f"<Team {self.name}>"
```

app/models/project.py (67 lineas)

Proyectos y sus propiedades calculadas (progress, is_delayed...).

```
from datetime import datetime, date

from ..extensions import db
from .enums import ProjectStatus, Priority, TaskStatus

class Project(db.Model):
    __tablename__ = "projects"

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(150), nullable=False)
    description = db.Column(db.Text)
    status = db.Column(db.String(30), default=ProjectStatus.PLANNING, nullable=False, index=True)
    priority = db.Column(db.String(20), default=Priority.MEDIUM, nullable=False)
    start_date = db.Column(db.Date)
    end_date = db.Column(db.Date)
    manager_id = db.Column(db.Integer, db.ForeignKey("users.id"))
    team_id = db.Column(db.Integer, db.ForeignKey("teams.id"))
    created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

    manager = db.relationship("User", back_populates="managed_projects", foreign_keys=[manager_id])
    team = db.relationship("Team", back_populates="projects")
    tasks = db.relationship(
        "Task",
        back_populates="project",
        cascade="all, delete-orphan",
        order_by="Task.start_date",
    )

    # --- Metricas derivadas ---
    @property
    def progress(self):
        if not self.tasks:
            return 0
        done = sum(1 for t in self.tasks if t.status == TaskStatus.DONE)
        return round(done / len(self.tasks) * 100)

    @property
    def task_count(self):
        return len(self.tasks)

    @property
    def open_task_count(self):
        return sum(1 for t in self.tasks if t.status != TaskStatus.DONE)

    @property
```

```

def overdue_task_count(self):
    return sum(1 for t in self.tasks if t.is_overdue)

@property
def is_delayed(self):
    return (
        self.end_date is not None
        and self.end_date < date.today()
        and self.status not in ProjectStatus.CLOSED
    )

@property
def status_label(self):
    return ProjectStatus.LABELS.get(self.status, self.status)

@property
def priority_label(self):
    return Priority.LABELS.get(self.priority, self.priority)

def __repr__(self):
    return f"<Project {self.name}>"

```

app/models/task.py (92 lines)

La entidad central: tareas, su relacion N:M auto-referencial de dependencias, y to_gantt() que arma el JSON para Frappe Gantt.

```

from datetime import datetime, date

from ..extensions import db
from .enums import TaskStatus, Priority

# Dependencias entre tareas (para el diagrama de Gantt)
task_dependencies = db.Table(
    "task_dependencies",
    db.Column("task_id", db.Integer, db.ForeignKey("tasks.id", ondelete="CASCADE"), primary_key=True),
    db.Column("depends_on_id", db.Integer, db.ForeignKey("tasks.id", ondelete="CASCADE"), primary_key=True),
)

class Task(db.Model):
    __tablename__ = "tasks"

    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(200), nullable=False)
    description = db.Column(db.Text)
    status = db.Column(db.String(30), default=TaskStatus.TODO, nullable=False, index=True)
    priority = db.Column(db.String(20), default=Priority.MEDIUM, nullable=False)
    progress = db.Column(db.Integer, default=0, nullable=False)
    estimated_hours = db.Column(db.Float)
    start_date = db.Column(db.Date)
    due_date = db.Column(db.Date, index=True)
    completed_at = db.Column(db.DateTime)

    project_id = db.Column(
        db.Integer, db.ForeignKey("projects.id", ondelete="CASCADE"), nullable=False, index=True
    )
    assignee_id = db.Column(db.Integer, db.ForeignKey("users.id"), index=True)
    created_by_id = db.Column(db.Integer, db.ForeignKey("users.id"))
    created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

    project = db.relationship("Project", back_populates="tasks")
    assignee = db.relationship("User", back_populates="assigned_tasks", foreign_keys=[assignee_id])
    creator = db.relationship("User", foreign_keys=[created_by_id])
    comments = db.relationship(
        "Comment", back_populates="task", cascade="all, delete-orphan", order_by="Comment.created_at"
    )
    dependencies = db.relationship(
        "Task",
        secondary=task_dependencies,
        primaryjoin="Task.id == task_dependencies.c.task_id",
        secondaryjoin="Task.id == task_dependencies.c.depends_on_id",
        backref="dependents",
    )

    # --- Estado temporal ---
    @property
    def is_overdue(self):
        return bool(self.due_date and self.due_date < date.today() and self.status != TaskStatus.DONE)

    @property
    def days_to_due(self):

```

```

        return (self.due_date - date.today()).days if self.due_date else None

def is_due_soon(self, window=3):
    d = self.days_to_due
    return d is not None and 0 <= d <= window and self.status != TaskStatus.DONE

@property
def status_label(self):
    return TaskStatus.LABELS.get(self.status, self.status)

@property
def priority_label(self):
    return Priority.LABELS.get(self.priority, self.priority)

def mark_done(self):
    self.status = TaskStatus.DONE
    self.progress = 100
    self.completed_at = datetime.utcnow()

# --- Serializacion para Frappe Gantt ---
def to_gantt(self):
    start = self.start_date or self.due_date or date.today()
    end = self.due_date or self.start_date or date.today()
    if end < start:
        end = start
    return {
        "id": str(self.id),
        "name": self.title,
        "start": start.isoformat(),
        "end": end.isoformat(),
        "progress": self.progress or 0,
        "dependencies": ", ".join(str(d.id) for d in self.dependencies),
        "custom_class": f"bar-{self.status}",
    }

def __repr__(self):
    return f"<Task {self.title!r}>"

```

app/models/comment.py (21 lines)

Comentarios de una tarea.

```

from datetime import datetime

from ..extensions import db

class Comment(db.Model):
    __tablename__ = "comments"

    id = db.Column(db.Integer, primary_key=True)
    body = db.Column(db.Text, nullable=False)
    task_id = db.Column(
        db.Integer, db.ForeignKey("tasks.id", ondelete="CASCADE"), nullable=False, index=True
    )
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"), nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

    task = db.relationship("Task", back_populates="comments")
    author = db.relationship("User", back_populates="comments")

    def __repr__(self):
        return f"<Comment task={self.task_id}>"

```

Avisos por usuario, con dedup_key para no duplicarlos.

```
from datetime import datetime

from ..extensions import db

class Notification(db.Model):
    __tablename__ = "notifications"

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(
        db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"), nullable=False, index=True
    )
    message = db.Column(db.String(255), nullable=False)
    link = db.Column(db.String(255))
    category = db.Column(db.String(30), default="info") # info | warning | danger | success
    is_read = db.Column(db.Boolean, default=False, nullable=False)
    dedup_key = db.Column(db.String(120), index=True) # evita duplicados del scanner
    created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

    user = db.relationship("User", back_populates="notifications")

    def __repr__(self):
        return f"<Notification u={self.user_id} {self.message[:20]!r}>"
```

6.3 Formularios (Flask-WTF)

app/forms/auth_forms.py (42 lineas)

Formularios de login, registro y recuperacion, con las reglas de contraseña (_pwd_rules) que se explican en la seccion 2.

```
from flask_wtf import FlaskForm
from wtforms import BooleanField, PasswordField, StringField, SubmitField
from wtforms.validators import DataRequired, Email, EqualTo, Length, Regexp

_pwd_rules = [
    DataRequired(),
    Length(min=8, message="Minimo 8 caracteres."),
    Regexp(r"^[A-Za-z].*", message="Debe incluir al menos una letra."),
    Regexp(r"^[0-9].*", message="Debe incluir al menos un numero."),
]

class LoginForm(FlaskForm):
    email = StringField("Correo", validators=[DataRequired(), Email()])
    password = PasswordField("Contraseña", validators=[DataRequired()])
    remember = BooleanField("Recordarme")
    submit = SubmitField("Iniciar sesion")

class RegisterForm(FlaskForm):
    name = StringField("Nombre completo", validators=[DataRequired(), Length(2, 120)])
    email = StringField("Correo", validators=[DataRequired(), Email(), Length(max=120)])
    password = PasswordField("Contraseña", validators=_pwd_rules)
    confirm = PasswordField(
        "Confirmar contraseña",
        validators=[DataRequired(), EqualTo("password", message="Las contraseñas no coinciden.")],
    )
    submit = SubmitField("Crear cuenta")

class ForgotPasswordForm(FlaskForm):
    email = StringField("Correo", validators=[DataRequired(), Email()])
    submit = SubmitField("Enviar enlace de recuperacion")

class ResetPasswordForm(FlaskForm):
    password = PasswordField("Nueva contraseña", validators=_pwd_rules)
    confirm = PasswordField(
        "Confirmar contraseña",
        validators=[DataRequired(), EqualTo("password", message="Las contraseñas no coinciden.")],
    )
    submit = SubmitField("Restablecer contraseña")
```

app/forms/project_forms.py (21 lineas)

Formulario de proyecto, con validate_end_date para que no termine antes de empezar.

```
from flask_wtf import FlaskForm
from wtforms import (DateField, SelectField, StringField, SubmitField, TextAreaField)
from wtforms.validators import DataRequired, Length, Optional, ValidationError

from ..models.enums import ProjectStatus, Priority

class ProjectForm(FlaskForm):
    name = StringField("Nombre del proyecto", validators=[DataRequired(), Length(2, 150)])
    description = TextAreaField("Descripción", validators=[Optional(), Length(max=4000)])
    status = SelectField("Estado", choices=ProjectStatus.choices(), validators=[DataRequired()])
    priority = SelectField("Prioridad", choices=Priority.choices(), validators=[DataRequired()])
    start_date = DateField("Fecha de inicio", validators=[Optional()])
    end_date = DateField("Fecha de fin", validators=[Optional()])
    manager_id = SelectField("Gerente responsable", coerce=int, validators=[Optional()])
    team_id = SelectField("Equipo", coerce=int, validators=[Optional()])
    submit = SubmitField("Guardar proyecto")

    def validate_end_date(self, field):
        if field.data and self.start_date.data and field.data < self.start_date.data:
            raise ValidationError("La fecha de fin no puede ser anterior a la de inicio.")
```

app/forms/task_forms.py (31 líneas)

Formulario de tarea, incluida la lista de dependencias (SelectMultipleField).

```
from flask_wtf import FlaskForm
from wtforms import (DateField, DecimalField, IntegerField, SelectField,
                     SelectMultipleField, StringField, SubmitField, TextAreaField)
from wtforms.validators import (DataRequired, Length, NumberRange, Optional,
                               ValidationError)

from ..models.enums import TaskStatus, Priority

class TaskForm(FlaskForm):
    title = StringField("Titulo", validators=[DataRequired(), Length(2, 200)])
    description = TextAreaField("Descripcion", validators=[Optional(), Length(max=4000)])
    project_id = SelectField("Proyecto", coerce=int, validators=[DataRequired()])
    assignee_id = SelectField("Responsable", coerce=int, validators=[Optional()])
    status = SelectField("Estado", choices=TaskStatus.choices(), validators=[DataRequired()])
    priority = SelectField("Prioridad", choices=Priority.choices(), validators=[DataRequired()])
    progress = IntegerField("% Avance", default=0, validators=[Optional(), NumberRange(0, 100)])
    estimated_hours = DecimalField("Horas estimadas", places=1, validators=[Optional(), NumberRange(min=0)])
    start_date = DateField("Fecha de inicio", validators=[Optional()])
    due_date = DateField("Fecha de vencimiento", validators=[Optional()])
    dependencies = SelectMultipleField("Depende de", coerce=int, validators=[Optional()])
    submit = SubmitField("Guardar tarea")

    def validate_due_date(self, field):
        if field.data and self.start_date.data and field.data < self.start_date.data:
            raise ValidationError("El vencimiento no puede ser anterior al inicio.")

class QuickCommentForm(FlaskForm):
    body = TextAreaField("Comentario", validators=[DataRequired(), Length(1, 2000)])
    submit = SubmitField("Comentar")
```

app/forms/team_forms.py (10 líneas)

Formulario de equipo (nombre, descripcion, integrantes).

```
from flask_wtf import FlaskForm
from wtforms import SelectMultipleField, StringField, SubmitField, TextAreaField
from wtforms.validators import DataRequired, Length, Optional

class TeamForm(FlaskForm):
    name = StringField("Nombre del equipo", validators=[DataRequired(), Length(2, 120)])
    description = TextAreaField("Descripcion", validators=[Optional(), Length(max=2000)])
    member_ids = SelectMultipleField("Integrantes", coerce=int, validators=[Optional()])
    submit = SubmitField("Guardar equipo")
```

Formulario de usuario (alta/edición por un admin) y el formulario de perfil propio.

```
from flask_wtf import FlaskForm
from wtforms import (BooleanField, PasswordField, SelectField, StringField,
                     SubmitField)
from wtforms.validators import DataRequired, Email, Length, Optional

class UserForm(FlaskForm):
    name = StringField("Nombre completo", validators=[DataRequired(), Length(2, 120)])
    email = StringField("Correo", validators=[DataRequired(), Email(), Length(max=120)])
    job_title = StringField("Cargo", validators=[Optional(), Length(max=120)])
    role_id = SelectField("Rol", coerce=int, validators=[DataRequired()])
    is_active = BooleanField("Cuenta activa", default=True)
    password = PasswordField(
        "Contraseña", validators=[Optional(), Length(min=8, message="Mínimo 8 caracteres.")]
    )
    submit = SubmitField("Guardar usuario")

class ProfileForm(FlaskForm):
    name = StringField("Nombre completo", validators=[DataRequired(), Length(2, 120)])
    job_title = StringField("Cargo", validators=[Optional(), Length(max=120)])
    avatar_url = StringField("URL de avatar", validators=[Optional(), Length(max=255)])
    current_password = PasswordField("Contraseña actual", validators=[Optional()])
    new_password = PasswordField("Nueva contraseña", validators=[Optional(), Length(min=8)])
    submit = SubmitField("Actualizar perfil")
```

6.4 Utilidades (permisos, alcance, modales)

app/utls/decorators.py (30 lineas)

Decoradores de ruta que cortan el acceso segun el rol antes de ejecutar la vista.

```
"""Decoradores de autorizacion por rol."""
from functools import wraps

from flask import abort
from flask_login import current_user

from ..models import Role


def role_required(*roles):
    def decorator(view):
        @wraps(view)
        def wrapper(*args, **kwargs):
            if not current_user.is_authenticated:
                abort(401)
            if not current_user.has_role(*roles):
                abort(403)
            return view(*args, **kwargs)

        return wrapper

    return decorator


def admin_required(view):
    return role_required(Role.ADMIN)(view)


def manager_required(view):
    return role_required(Role.ADMIN, Role.MANAGER)(view)
```

app/utls/permissions.py (32 lineas)

Funciones puras que deciden si un usuario puede ver/editar/borrar un proyecto o tarea concretos.

```
"""Reglas de acceso a nivel de objeto."""


def can_view_project(user, project):
    if user.is_admin:
        return True
    if user.is_manager and project.manager_id == user.id:
        return True
    if project.team and user in project.team.members:
        return True
    return any(t.assignee_id == user.id for t in project.tasks)


def can_edit_project(user, project):
    return user.is_admin or (user.is_manager and project.manager_id == user.id)


def can_delete_project(user, project):
    return can_edit_project(user, project)


def can_edit_task(user, task):
    return (
        user.is_admin
        or (user.is_manager and task.project.manager_id == user.id)
        or task.assignee_id == user.id
    )


def can_manage_task(user, task):
    """Crear / borrar / reasignar (no solo cambiar estado)."""
    return user.is_admin or (user.is_manager and task.project.manager_id == user.id)
```

app/utills/scope.py (30 lineas)

Filtra las consultas segun el rol para que la base de datos ni siquiera devuelva registros que no le corresponden al usuario.

```
"""Consultas filtradas segun el rol del usuario autenticado."""
from ..extensions import db
from ..models import Project, Task, Team, User

def projects_for(user):
    q = Project.query
    if user.is_admin:
        return q
    if user.is_manager:
        return q.filter(Project.manager_id == user.id)
    return q.filter(
        db.or_(
            Project.tasks.any(Task.assignee_id == user.id),
            Project.team.has(Team.members.any(User.id == user.id)),
        )
    )

def tasks_for(user):
    q = Task.query
    if user.is_admin:
        return q
    if user.is_manager:
        return q.join(Project).filter(Project.manager_id == user.id)
    return q.filter(Task.assignee_id == user.id)

def visible_project_ids(user):
    return [p.id for p in projects_for(user).with_entities(Project.id).all()]
```

app/utills/http.py (41 lineas)

El helper del patron de modales AJAX: decide si responder JSON/fragmento o la pagina completa de siempre.

```
"""Helpers para servir formularios en modal (AJAX) o pagina completa (fallback)."""
from flask import flash, jsonify, redirect, render_template, request

def wants_json():
    """True cuando la vista debe responder JSON / fragmento en vez de pagina completa.

    Lo activan las peticiones AJAX de los modales (cabecera ``X-Requested-With``)
    o un ``Accept: application/json`` explicito.
    """
    if request.headers.get("X-Requested-With") == "XMLHttpRequest":
        return True
    accept = request.accept_mimetypes
    return bool(accept.accept_json and not accept.accept_html)

def modal_success(redirect_url):
    """Exito al guardar: JSON con destino si es AJAX, redirect normal si no."""
    if wants_json():
        return jsonify(ok=True, redirect=redirect_url)
    return redirect(redirect_url)

def modal_blocked(message, redirect_url):
    """Accion no permitida (p. ej. eliminar con dependencias)."""
    if wants_json():
        return jsonify(ok=False, error=message)
    flash(message, "danger")
    return redirect(redirect_url)

def modal_form(template, form, **context):
    """Fragmento del formulario para el modal (GET) o con errores (POST invalido).

    Devuelve ``None`` cuando la peticion NO es AJAX, para que la vista haga el
    render de la pagina completa como siempre.
    """
    if wants_json():
        status = 422 if form.errors else 200
        return render_template(template, form=form, **context), status
    return None
```


6.5 Servicios (logica de negocio)

app/services/report_service.py (137 lineas)

Convierte proyectos y tareas en las estructuras {labels, datasets} que necesita Chart.js, con una paleta de color consistente en toda la app.

```
"""Agregaciones para el dashboard y el modulo de reportes.

Devuelve estructuras listas para Chart.js: {labels: [...], datasets: [...]}.
"""
from collections import Counter, defaultdict
from datetime import date

from ..models import Project, Task, User
from ..models.enums import ProjectStatus, TaskStatus, Priority

PALETTE = ["#2563eb", "#7c3aed", "#10b981", "#f97316", "#ef4444", "#0ea5e9", "#ec4899"]

def dashboard_stats(projects, tasks):
    today = date.today()
    return {
        "total_projects": len(projects),
        "active_projects": sum(1 for p in projects if p.status == ProjectStatus.ACTIVE),
        "delayed_projects": sum(1 for p in projects if p.is_delayed),
        "total_tasks": len(tasks),
        "pending_tasks": sum(1 for t in tasks if t.status != TaskStatus.DONE),
        "overdue_tasks": sum(1 for t in tasks if t.is_overdue),
        "due_soon_tasks": sum(1 for t in tasks if t.is_due_soon()),
        "done_tasks": sum(1 for t in tasks if t.status == TaskStatus.DONE),
    }

def project_progress_chart(projects):
    projects = sorted(projects, key=lambda p: p.progress, reverse=True)[:12]
    return {
        "labels": [p.name for p in projects],
        "datasets": [{
            "label": "% completado",
            "data": [p.progress for p in projects],
            "backgroundColor": PALETTE[0],
        }],
    }

def task_status_chart(tasks):
    counts = Counter(t.status for t in tasks)
    labels = [TaskStatus.LABELS[s] for s in TaskStatus.ALL]
    return {
        "labels": labels,
        "datasets": [{
            "data": [counts.get(s, 0) for s in TaskStatus.ALL],
            "backgroundColor": PALETTE[4],
        }],
    }

def workload_chart(tasks):
    """Tareas abiertas por usuario asignado."""
    by_user = defaultdict(int)
    for t in tasks:
        if t.status != TaskStatus.DONE and t.assignee:
            by_user[t.assignee.name] += 1
    items = sorted(by_user.items(), key=lambda kv: kv[1], reverse=True)[:12]
    return {
        "labels": [name for name, _ in items],
        "datasets": [{
            "label": "Tareas abiertas",
            "data": [n for _, n in items],
            "backgroundColor": PALETTE[1],
        }],
    }

def deadline_compliance_chart(tasks):
    """Cumplimiento de plazos sobre tareas completadas."""
    on_time = late = 0
    for t in tasks:
        if t.status == TaskStatus.DONE and t.due_date and t.completed_at:
```

```

        if t.completed_at.date() <= t.due_date:
            on_time += 1
        else:
            late += 1
    pending_overdue = sum(1 for t in tasks if t.is_overdue)
    return {
        "labels": ["A tiempo", "Con retraso", "Vencidas sin cerrar"],
        "datasets": [{
            "data": [on_time, late, pending_overdue],
            "backgroundColor": [PALETTE[2], PALETTE[3], PALETTE[4]],
        }],
    }

def priority_distribution_chart(tasks):
    counts = Counter(t.priority for t in tasks if t.status != TaskStatus.DONE)
    return {
        "labels": [Priority.LABELS[p] for p in Priority.ALL],
        "datasets": [{
            "data": [counts.get(p, 0) for p in Priority.ALL],
            "backgroundColor": [PALETTE[2], PALETTE[1], PALETTE[3], PALETTE[4]],
        }],
    }

def monthly_completion_chart(tasks):
    """Serie de tareas completadas por mes (ultimos 6 meses)."""
    from datetime import date as _d

    today = _d.today()
    months = []
    for i in range(5, -1, -1):
        y = today.year + (today.month - 1 - i) // 12
        m = (today.month - 1 - i) % 12 + 1
        months.append((y, m))
    counts = {ym: 0 for ym in months}
    for t in tasks:
        if t.status == TaskStatus.DONE and t.completed_at:
            ym = (t.completed_at.year, t.completed_at.month)
            if ym in counts:
                counts[ym] += 1
    labels = [f"{m:02d}/{y}" for y, m in months]
    return {
        "labels": labels,
        "datasets": [{
            "label": "Tareas completadas",
            "data": [counts[ym] for ym in months],
            "borderColor": PALETTE[0],
            "backgroundColor": "rgba(37, 99, 235, .15)",
            "fill": True,
            "tension": 0.35,
        }],
    }

def build_report_bundle(projects, tasks):
    return {
        "progress": project_progress_chart(projects),
        "status": task_status_chart(tasks),
        "workload": workload_chart(tasks),
        "compliance": deadline_compliance_chart(tasks),
        "priority": priority_distribution_chart(tasks),
        "monthly": monthly_completion_chart(tasks),
    }

```

app/services/export_service.py (275 lineas)

Genera el Excel (openpyxl) y los dos PDF con ReportLab: el reporte general y el Gantt vectorial por proyecto (el mismo motor que genera este documento).

```

"""Exportacion de reportes a Excel (.xlsx) y PDF."""
from io import BytesIO
from datetime import datetime, date, timedelta

from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment
from reportlab.lib import colors
from reportlab.lib.colors import Color, HexColor
from reportlab.lib.pagesizes import A4, landscape
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.lib.units import cm

```

```

from reportlab.graphics.shapes import Drawing, Line, Rect, String
from reportlab.platypus import (PageBreak, Paragraph, SimpleDocTemplate, Spacer,
                                Table, TableStyle)

from ..models.enums import TaskStatus

HEADER_FILL = PatternFill("solid", fgColor="4F46E5")
HEADER_FONT = Font(color="FFFFFF", bold=True)

GANTT_STATUS_COLORS = {
    TaskStatus.TODO: HexColor("#94a3b8"),
    TaskStatus.IN_PROGRESS: HexColor("#2563eb"),
    TaskStatus.REVIEW: HexColor("#f97316"),
    TaskStatus.DONE: HexColor("#10b981"),
}

def _fmt_date(d):
    return d.strftime("%d/%m/%Y") if d else "-"

# ----- #
# EXCEL
# ----- #
def projects_to_excel(projects):
    wb = Workbook()
    ws = wb.active
    ws.title = "Proyectos"
    headers = ["ID", "Nombre", "Estado", "Prioridad", "Gerente", "Equipo",
               "Inicio", "Fin", "% Avance", "Tareas", "Abiertas", "Vencidas"]
    ws.append(headers)
    for c in ws[1]:
        c.fill, c.font, c.alignment = HEADER_FILL, HEADER_FONT, Alignment(horizontal="center")

    for p in projects:
        ws.append([
            p.id, p.name, p.status_label, p.priority_label,
            p.manager.name if p.manager else "-",
            p.team.name if p.team else "-",
            _fmt_date(p.start_date), _fmt_date(p.end_date),
            p.progress, p.task_count, p.open_task_count, p.overdue_task_count,
        ])

    # Hoja de tareas
    ws2 = wb.create_sheet("Tareas")
    h2 = ["ID", "Proyecto", "Tarea", "Estado", "Prioridad", "Responsable",
          "Inicio", "Vence", "% Avance", "Vencida"]
    ws2.append(h2)
    for c in ws2[1]:
        c.fill, c.font, c.alignment = HEADER_FILL, HEADER_FONT, Alignment(horizontal="center")
    for p in projects:
        for t in p.tasks:
            ws2.append([
                t.id, p.name, t.title, t.status_label, t.priority_label,
                t.assignee.name if t.assignee else "-",
                _fmt_date(t.start_date), _fmt_date(t.due_date),
                t.progress, "Si" if t.is_overdue else "No",
            ])

    for sheet in (ws, ws2):
        for col in sheet.columns:
            width = max((len(str(c.value)) for c in col if c.value is not None), default=10)
            sheet.column_dimensions[col[0].column_letter].width = min(width + 3, 45)

    buffer = BytesIO()
    wb.save(buffer)
    buffer.seek(0)
    return buffer

# ----- #
# PDF
# ----- #
def report_to_pdf(projects, stats):
    buffer = BytesIO()
    doc = SimpleDocTemplate(
        buffer, pagesize=landscape(A4),
        topMargin=1.5 * cm, bottomMargin=1.5 * cm,
        leftMargin=1.2 * cm, rightMargin=1.2 * cm,
    )
    styles = getSampleStyleSheet()
    elems = [
        Paragraph("Reporte de Proyectos - GestorPro", styles["Title"]),
    ]

```

```

Paragraph(datetime.now().strftime("Generado el %d/%m/%Y %H:%M"), styles["Normal"]),
Spacer(1, 0.6 * cm),
]

resumen = [
    ["Proyectos", "Activos", "Retrasados", "Tareas pendientes", "Tareas vencidas"],
    [stats["total_projects"], stats["active_projects"], stats["delayed_projects"],
     stats["pending_tasks"], stats["overdue_tasks"]],
]
t = Table(resumen, hAlign="LEFT")
t.setStyle(TableStyle([
    ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#4F46E5")),
    ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
    ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
    ("GRID", (0, 0), (-1, -1), 0.5, colors.grey),
    ("ALIGN", (0, 0), (-1, -1), "CENTER"),
    ("FONTSIZE", (0, 1), (-1, 1), 13),
    ("TOPPADDING", (0, 0), (-1, -1), 6),
    ("BOTTOMPADDING", (0, 0), (-1, -1), 6),
]))
elems += [t, Spacer(1, 0.8 * cm),
          Paragraph("Detalle de proyectos", styles["Heading2"])]

data = [["Nombre", "Estado", "Prioridad", "Gerente", "Inicio", "Fin", "% Avance", "Tareas"]]
for p in projects:
    data.append([
        Paragraph(p.name, styles["BodyText"]), p.status_label, p.priority_label,
        p.manager.name if p.manager else "-",
        _fmt_date(p.start_date), _fmt_date(p.end_date),
        f"{p.progress}%", p.task_count,
    ])
dt = Table(data, repeatRows=1, colWidths=[6 * cm, 2.6 * cm, 2.2 * cm, 4 * cm,
                                           2.6 * cm, 2.6 * cm, 2 * cm, 1.8 * cm])
dt.setStyle(TableStyle([
    ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#1e293b")),
    ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
    ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
    ("GRID", (0, 0), (-1, -1), 0.4, colors.HexColor("#cbd5e1")),
    ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#f1f5f9")]),
    ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
    ("FONTSIZE", (0, 1), (-1, -1), 9),
]))
elems.append(dt)

doc.build(elems)
buffer.seek(0)
return buffer

# ----- #
# PDF - Cronograma (Gantt) por proyecto
# ----- #
def _task_range(t):
    start = t.start_date or t.due_date or date.today()
    end = t.due_date or t.start_date or date.today()
    return start, (end if end >= start else start)

def _darken(c, f=0.55):
    return Color(c.red * f, c.green * f, c.blue * f)

def _ganttt_legend():
    d = Drawing(700, 14)
    x = 0
    for st in TaskStatus.ALL:
        label = TaskStatus.LABELS[st]
        d.add(Rect(x, 2, 10, 10, fillColor=GANTT_STATUS_COLORS[st], strokeColor=None))
        d.add(String(x + 14, 3, label, fontSize=7, fillColor=HexColor("#334155")))
        x += 22 + 4.6 * len(label) + 14
    return d

def _ganttt_drawing(tasks, width, dmin, dmax, row_h=19, label_w=175):
    total_days = max((dmax - dmin).days, 1)
    chart_w = width - label_w
    height = row_h * len(tasks) + 26
    top = height - 22
    d = Drawing(width, height)

    def x_of(dt):
        return label_w + (dt - dmin).days / total_days * chart_w
    # --- Ejes de tiempo (semanal si el rango es corto, mensual si es largo) ---

```

```

ticks = []
if total_days <= 70:
    cur = dmin - timedelta(days=dmin.weekday())
    while cur <= dmax:
        if cur >= dmin:
            ticks.append((cur, cur.strftime("%d/%m")))
        cur += timedelta(days=7)
else:
    cur = date(dmin.year, dmin.month, 1)
    while cur <= dmax:
        if cur >= dmin:
            ticks.append((cur, cur.strftime("%b %Y")))
        year, month = divmod(cur.month, 12)
        cur = date(cur.year + year, month + 1, 1)

for tk, text in ticks:
    xx = x_of(tk)
    d.add(Line(xx, 0, xx, top, strokeColor=HexColor("#e2e8f0"), strokeWidth=0.5))
    d.add(String(xx + 2, top + 6, text, fontSize=6.5, fillColor=HexColor("#64748b")))

today = date.today()
if dmin <= today <= dmax:
    xx = x_of(today)
    d.add(Line(xx, 0, xx, top + 2, strokeColor=HexColor("#ef4444"),
        strokeWidth=0.9, strokeDashArray=[2, 2]))
    d.add(String(xx + 2, 1, "hoy", fontSize=6, fillColor=HexColor("#ef4444")))

d.add(Line(label_w, 0, label_w, top, strokeColor=HexColor("#cbd5e1"), strokeWidth=0.6))

for i, t in enumerate(tasks):
    y = top - (i + 1) * row_h
    start, end = _task_range(t)
    x1, x2 = x_of(start), x_of(end)
    bar_w = max(x2 - x1, 2.5)
    color = GANTT_STATUS_COLORS.get(t.status, HexColor("#94a3b8"))

    name = t.title if len(t.title) <= 33 else t.title[:32] + "..."
    d.add(String(4, y + 6, name, fontSize=7, fillColor=HexColor("#1e293b")))
    d.add(Rect(label_w, y + 3, chart_w, row_h - 7,
        fillColor=HexColor("#f1f5f9"), strokeColor=None))
    d.add(Rect(x1, y + 3, bar_w, row_h - 7, rx=2, ry=2,
        fillColor=color, strokeColor=None))
    pct = max(0, min(t.progress or 0, 100))
    if pct:
        d.add(Rect(x1, y + 3, bar_w * pct / 100.0, row_h - 7, rx=2, ry=2,
            fillColor=_darken(color), strokeColor=None))
    d.add(String(min(x1 + bar_w + 3, width - 22), y + 6, f"{pct}%",
        fontSize=6, fillColor=HexColor("#64748b")))

return d

def project_gantt_to_pdf(project):
    buffer = BytesIO()
    page_w = landscape(A4)[0]
    doc = SimpleDocTemplate(
        buffer, pagesize=landscape(A4),
        topMargin=1.3 * cm, bottomMargin=1.3 * cm,
        leftMargin=1.2 * cm, rightMargin=1.2 * cm,
    )
    styles = getSampleStyleSheet()
    usable_w = page_w - 2.4 * cm

    elems = [
        Paragraph(f"Cronograma (Gantt) - {project.name}", styles["Title"]),
        Paragraph(datetime.now().strftime("Generado el %d/%m/%Y %H:%M"), styles["Normal"]),
    ]

    tasks = sorted(
        project.tasks,
        key=lambda t: (_task_range(t)[0], t.id),
    )
    if not tasks:
        elems.append(Spacer(1, 0.6 * cm))
        elems.append(Paragraph("Este proyecto todavia no tiene tareas.", styles["Normal"]))
        doc.build(elems)
        buffer.seek(0)
        return buffer

    dmin = min(_task_range(t)[0] for t in tasks)
    dmax = max(_task_range(t)[1] for t in tasks)
    if dmin == dmax:
        dmax = dmin + timedelta(days=7)
    pad = max(1, round((dmax - dmin).days * 0.04))

```

```

dmin -= timedelta(days=pad)
dmax += timedelta(days=pad)

elems += [Spacer(1, 0.25 * cm), _gantt_legend(), Spacer(1, 0.2 * cm)]

per_page = 18
for i in range(0, len(tasks), per_page):
    elems.append(_gantt_drawing(tasks[i:i + per_page], usable_w, dmin, dmax))
    if i + per_page < len(tasks):
        elems.append(PageBreak())

doc.build(elems)
buffer.seek(0)
return buffer

```

app/services/notification_service.py (67 líneas)

Crea notificaciones evitando duplicados y el escaner de vencimientos pensado para un cron.

```

"""Generacion y consulta de notificaciones."""
from datetime import date, timedelta

from flask import current_app

from ..extensions import db
from ..models import Notification, Task
from ..models.enums import TaskStatus

def notify(user, message, link=None, category="info", dedup_key=None):
    if dedup_key and Notification.query.filter_by(user_id=user.id, dedup_key=dedup_key).first():
        return None
    n = Notification(
        user_id=user.id, message=message, link=link, category=category, dedup_key=dedup_key
    )
    db.session.add(n)
    return n

def unread_count(user):
    return Notification.query.filter_by(user_id=user.id, is_read=False).count()

def mark_read(user, notification_id=None):
    q = Notification.query.filter_by(user_id=user.id, is_read=False)
    if notification_id:
        q = q.filter_by(id=notification_id)
    updated = q.update({Notification.is_read: True})
    db.session.commit()
    return updated

def scan_due_soon():
    """Crea notificaciones para tareas por vencer o vencidas. Ideal para un cron."""
    window = current_app.config.get("TASK_DUE_SOON_DAYS", 3)
    today = date.today()
    limit = today + timedelta(days=window)
    created = 0

    tasks = (
        Task.query.filter(
            Task.status != TaskStatus.DONE,
            Task.assignee_id.isnot(None),
            Task.due_date.isnot(None),
            Task.due_date <= limit,
        ).all()
    )
    for t in tasks:
        overdue = t.due_date < today
        key = f"due:{t.id}:{'overdue' if overdue else t.due_date.isoformat()}"
        msg = (
            f"La tarea '{t.title}' esta vencida ({t.due_date:%d/%m})"
            if overdue
            else f"La tarea '{t.title}' vence el {t.due_date:%d/%m}"
        )
        n = notify(
            t.assignee,
            msg,
            link=f"/tareass/{t.id}",
            category="danger" if overdue else "warning",
            dedup_key=key,

```

```

    )
    if n is not None:
        created += 1
    db.session.commit()
    return created

```

app/services/mail_service.py (31 lineas)

Envio del correo de recuperacion de contraseña.

```

"""Envio de correo. Si SMTP no esta configurado, se registra en consola."""
from flask import current_app, render_template, url_for
from flask_mail import Message

from ..extensions import mail

def _send(subject, recipients, body_text, body_html=None):
    app = current_app._get_current_object()
    if not app.config.get("MAIL_USERNAME") and not app.config.get("TESTING"):
        app.logger.warning("MAIL no configurado. Correo a %s:\n%s", recipients, body_text)
        return False
    msg = Message(subject=subject, recipients=recipients, body=body_text, html=body_html)
    try:
        mail.send(msg)
        return True
    except Exception as exc: # pragma: no cover - depende del entorno SMTP
        app.logger.error("Fallo al enviar correo: %s", exc)
        return False

def send_password_reset_email(user, token):
    link = url_for("auth.reset_password", token=token, _external=True)
    text = (
        f"Hola {user.name},\n\n"
        f"Solicitaste restablecer tu contraseña en GestorPro.\n"
        f"Abre este enlace (valido 1 hora): {link}\n\n"
        f"Si no fuiste tu, ignora este mensaje."
    )
    html = render_template("auth/email_reset.html", user=user, link=link)
    return _send("Recuperacion de contraseña - GestorPro", [user.email], text, html)

```

6.6 Controladores (blueprints)

app/controllers/auth.py (100 lineas)

Login, registro (primer usuario = admin), logout y recuperacion de contraseña -el detalle esta en la seccion 2.

```
"""Autenticacion: login, registro, logout y recuperacion de contraseña."""
from datetime import datetime

from flask import (Blueprint, flash, redirect, render_template, request, url_for)
from flask_login import current_user, login_required, login_user, logout_user

from ..extensions import db
from ..forms.auth_forms import (ForgotPasswordForm, LoginForm, RegisterForm,
                                ResetPasswordForm)
from ..models import Role, User
from ..services.mail_service import send_password_reset_email

auth_bp = Blueprint("auth", __name__)

@auth_bp.route("/login", methods=["GET", "POST"])
def login():
    if current_user.is_authenticated:
        return redirect(url_for("dashboard.index"))

    form = LoginForm()
    if form.validate_on_submit():
        user = User.query.filter_by(email=form.email.data.lower().strip()).first()
        if user and user.check_password(form.password.data):
            if not user.is_active:
                flash("Tu cuenta esta desactivada. Contacta al administrador.", "danger")
                return redirect(url_for("auth.login"))
            login_user(user, remember=form.remember.data)
            user.last_login_at = datetime.utcnow()
            db.session.commit()
            nxt = request.args.get("next")
            if nxt and nxt.startswith("/"):
                return redirect(nxt)
            return redirect(url_for("dashboard.index"))
        flash("Correo o contraseña incorrectos.", "danger")
    return render_template("auth/login.html", form=form)

@auth_bp.route("/registro", methods=["GET", "POST"])
def register():
    if current_user.is_authenticated:
        return redirect(url_for("dashboard.index"))

    form = RegisterForm()
    if form.validate_on_submit():
        email = form.email.data.lower().strip()
        if User.query.filter_by(email=email).first():
            flash("Ya existe una cuenta con ese correo.", "warning")
            return render_template("auth/register.html", form=form)

        # El primer usuario del sistema es administrador; el resto, colaboradores.
        first = User.query.count() == 0
        role = Role.get(Role.ADMIN if first else Role.COLLABORATOR)
        user = User(name=form.name.data.strip(), email=email, role=role)
        user.set_password(form.password.data)
        db.session.add(user)
        db.session.commit()
        flash("Cuenta creada correctamente. Ya puedes iniciar sesion.", "success")
        return redirect(url_for("auth.login"))
    return render_template("auth/register.html", form=form)

@auth_bp.route("/logout")
@login_required
def logout():
    logout_user()
    flash("Sesion cerrada.", "info")
    return redirect(url_for("auth.login"))

@auth_bp.route("/recuperar", methods=["GET", "POST"])
def forgot_password():
    form = ForgotPasswordForm()
    if form.validate_on_submit():
        user = User.query.filter_by(email=form.email.data.lower().strip()).first()
```

```

        if user:
            token = user.generate_reset_token(ttl_hours=1)
            db.session.commit()
            send_password_reset_email(user, token)
            # Respuesta identica exista o no el correo (evita enumeracion de usuarios)
            flash("Si el correo existe, enviamos un enlace para restablecer la contrasena.", "info")
            return redirect(url_for("auth.login"))
        return render_template("auth/forgot_password.html", form=form)

@auth_bp.route("/restablecer/<token>", methods=["GET", "POST"])
def reset_password(token):
    user = User.query.filter_by(reset_token=token).first()
    if not user or not user.verify_reset_token(token):
        flash("El enlace es invalido o ha expirado.", "danger")
        return redirect(url_for("auth.forgot_password"))

    form = ResetPasswordForm()
    if form.validate_on_submit():
        user.set_password(form.password.data)
        user.clear_reset_token()
        db.session.commit()
        flash("Contrasena actualizada. Inicia sesion.", "success")
        return redirect(url_for("auth.login"))
    return render_template("auth/reset_password.html", form=form)

```

app/controllers/dashboard.py (44 lineas)

Arma los datos de la pantalla de inicio combinando report_service con el alcance del usuario.

```

"""Dashboard principal con tarjetas resumen y graficos."""
from flask import Blueprint, render_template
from flask_login import current_user, login_required

from ..models import Task
from ..models.enums import TaskStatus
from ..services import report_service
from ..utils.scope import projects_for, tasks_for

dashboard_bp = Blueprint("dashboard", __name__)

@login_required
@dashboard_bp.route("/")
def index():
    projects = projects_for(current_user).all()
    tasks = tasks_for(current_user).all()

    stats = report_service.dashboard_stats(projects, tasks)
    charts = {
        "progress": report_service.project_progress_chart(projects),
        "status": report_service.task_status_chart(tasks),
        "monthly": report_service.monthly_completion_chart(tasks),
    }

    my_open_tasks = (
        Task.query.filter(
            Task.assignee_id == current_user.id, Task.status != TaskStatus.DONE
        )
        .order_by(Task.due_date.is_(None), Task.due_date.asc())
        .limit(8)
        .all()
    )
    recent_projects = sorted(projects, key=lambda p: p.created_at, reverse=True)[:5]
    delayed = [t for t in tasks if t.is_overdue][:8]

    return render_template(
        "dashboard/index.html",
        stats=stats,
        charts=charts,
        my_open_tasks=my_open_tasks,
        recent_projects=recent_projects,
        delayed=delayed,
    )

```

app/controllers/projects.py (141 lineas)

CRUD de proyectos con el patron de modal AJAX, vista de detalle con el Gantt y exportacion a PDF.

```

"""CRUD de proyectos + vista de cronograma (Gantt)."""

```

```

import re
from datetime import datetime

from flask import (Blueprint, abort, flash, render_template, request,
                  send_file, url_for)
from flask_login import current_user, login_required

from ..extensions import db
from ..forms.project_forms import ProjectForm
from ..models import Project, Role, Team, User
from ..models.enums import ProjectStatus
from ..services import export_service
from ..utils.decorators import manager_required
from ..utils.http import modal_form, modal_success
from ..utils.permissions import (can_delete_project, can_edit_project,
                                can_view_project)
from ..utils.scope import projects_for

projects_bp = Blueprint("projects", __name__)

def _populate_choices(form):
    managers = User.query.join(Role).filter(
        Role.name.in_([Role.ADMIN, Role.MANAGER]), User.active.is_(True)
    ).order_by(User.name).all()
    form.manager_id.choices = [(0, "- Sin asignar -")] + [(u.id, u.name) for u in managers]
    form.team_id.choices = [(0, "- Sin equipo -")] + [
        (t.id, t.name) for t in Team.query.order_by(Team.name).all()
    ]

@projects_bp.route("/")
@login_required
def index():
    page = request.args.get("page", 1, type=int)
    status = request.args.get("status", "")
    q = request.args.get("q", "").strip()

    query = projects_for(current_user)
    if status in ProjectStatus.ALL:
        query = query.filter(Project.status == status)
    if q:
        query = query.filter(Project.name.ilike(f"%{q}%"))

    pagination = query.order_by(Project.created_at.desc()).paginate(
        page=page, per_page=12, error_out=False
    )
    return render_template(
        "projects/index.html", pagination=pagination, status=status, q=q
    )

@projects_bp.route("/<int:project_id>")
@login_required
def detail(project_id):
    project = Project.query.get_or_404(project_id)
    if not can_view_project(current_user, project):
        abort(403)
    gantt_tasks = [t.to_gantt() for t in project.tasks]
    return render_template(
        "projects/detail.html",
        project=project,
        gantt_tasks=gantt_tasks,
        can_edit=can_edit_project(current_user, project),
    )

@projects_bp.route("/<int:project_id>/gantt.pdf")
@login_required
def gantt_pdf(project_id):
    project = Project.query.get_or_404(project_id)
    if not can_view_project(current_user, project):
        abort(403)
    buf = export_service.project_gantt_to_pdf(project)
    slug = re.sub(r"^[^w\.-]+", "_", project.name).strip("_") or "proyecto"
    name = f"cronograma_{slug}_{datetime.now():%Y%m%d}.pdf"
    return send_file(buf, as_attachment=True, download_name=name,
                    mimetype="application/pdf")

@projects_bp.route("/nuevo", methods=["GET", "POST"])
@manager_required
def create():

```

```

form = ProjectForm()
_populate_choices(form)
if form.validate_on_submit():
    project = Project(
        name=form.name.data.strip(),
        description=form.description.data,
        status=form.status.data,
        priority=form.priority.data,
        start_date=form.start_date.data,
        end_date=form.end_date.data,
        manager_id=form.manager_id.data or None,
        team_id=form.team_id.data or None,
    )
    if not project.manager_id and current_user.is_manager:
        project.manager_id = current_user.id
    db.session.add(project)
    db.session.commit()
    flash("Proyecto creado.", "success")
    return modal_success(url_for("projects.detail", project_id=project.id))
return modal_form("projects/_form_modal.html", form, project=None) \
    or render_template("projects/form.html", form=form, project=None)

@projects_bp.route("/<int:project_id>/editar", methods=["GET", "POST"])
@login_required
def edit(project_id):
    project = Project.query.get_or_404(project_id)
    if not can_edit_project(current_user, project):
        abort(403)

    form = ProjectForm(obj=project)
    _populate_choices(form)
    if form.validate_on_submit():
        form.populate_obj(project)
        project.manager_id = form.manager_id.data or None
        project.team_id = form.team_id.data or None
        db.session.commit()
        flash("Proyecto actualizado.", "success")
        return modal_success(url_for("projects.detail", project_id=project.id))

    if request.method == "GET":
        form.manager_id.data = project.manager_id or 0
        form.team_id.data = project.team_id or 0
    return modal_form("projects/_form_modal.html", form, project=project) \
        or render_template("projects/form.html", form=form, project=project)

@projects_bp.route("/<int:project_id>/eliminar", methods=["POST"])
@login_required
def delete(project_id):
    project = Project.query.get_or_404(project_id)
    if not can_delete_project(current_user, project):
        abort(403)
    db.session.delete(project)
    db.session.commit()
    flash("Proyecto eliminado.", "info")
    return modal_success(url_for("projects.index"))

```

app/controllers/tasks.py (189 lineas)

CRUD de tareas, comentarios y cambio rapido de estado.

```

"""CRUD de tareas, cambio rapido de estado y comentarios."""
from datetime import datetime

from flask import (Blueprint, abort, flash, redirect, render_template, request,
                  url_for)
from flask_login import current_user, login_required

from ..extensions import db
from ..forms.task_forms import QuickCommentForm, TaskForm
from ..models import Comment, Project, Task, User
from ..models.enums import TaskStatus
from ..services.notification_service import notify
from ..utils.http import modal_form, modal_success
from ..utils.permissions import (can_edit_task, can_manage_task,
                                can_view_project)
from ..utils.scope import projects_for, tasks_for

tasks_bp = Blueprint("tasks", __name__)
def _editable_projects():

```

```

        return projects_for(current_user).order_by(Project.name).all()

def _populate_choices(form):
    projects = _editable_projects()
    form.project_id.choices = [(p.id, p.name) for p in projects]
    users = User.query.filter(User.active.is_(True)).order_by(User.name).all()
    form.assignee_id.choices = [(0, "- Sin asignar -")] + [(u.id, u.name) for u in users]
    task_opts = Task.query.order_by(Task.title).all()
    form.dependencies.choices = [(t.id, f"{t.title} ({t.project.name})") for t in task_opts]

@tasks_bp.route("/")
@login_required
def index():
    page = request.args.get("page", 1, type=int)
    status = request.args.get("status", "")
    project_id = request.args.get("project", type=int)
    mine = request.args.get("mine") == "1"

    query = tasks_for(current_user)
    if mine:
        query = query.filter(Task.assignee_id == current_user.id)
    if status in TaskStatus.ALL:
        query = query.filter(Task.status == status)
    if project_id:
        query = query.filter(Task.project_id == project_id)

    pagination = query.order_by(
        Task.due_date.is_(None), Task.due_date.asc()
    ).paginate(page=page, per_page=15, error_out=False)

    projects = projects_for(current_user).order_by(Project.name).all()
    return render_template(
        "tasks/index.html", pagination=pagination, projects=projects,
        status=status, project_id=project_id, mine=mine,
    )

@tasks_bp.route("/<int:task_id>", methods=["GET", "POST"])
@login_required
def detail(task_id):
    task = Task.query.get_or_404(task_id)
    if not can_view_project(current_user, task.project):
        abort(403)

    form = QuickCommentForm()
    if form.validate_on_submit():
        db.session.add(Comment(body=form.body.data, task_id=task.id, user_id=current_user.id))
        # avisar al responsable si comenta otra persona
        if task.assignee_id and task.assignee_id != current_user.id:
            notify(task.assignee, f"Nuevo comentario en '{task.title}'",
                link=url_for("tasks.detail", task_id=task.id))
        db.session.commit()
        flash("Comentario agregado.", "success")
        return redirect(url_for("tasks.detail", task_id=task.id))

    return render_template(
        "tasks/detail.html", task=task, form=form,
        can_edit=can_edit_task(current_user, task),
        can_manage=can_manage_task(current_user, task),
    )

@tasks_bp.route("/nueva", methods=["GET", "POST"])
@login_required
def create():
    if not current_user.has_role("admin", "gerente"):
        abort(403)
    form = TaskForm()
    _populate_choices(form)
    preselected = request.args.get("project", type=int)
    if preselected and request.method == "GET":
        form.project_id.data = preselected

    if form.validate_on_submit():
        task = Task(
            title=form.title.data.strip(),
            description=form.description.data,
            project_id=form.project_id.data,
            assignee_id=form.assignee_id.data or None,
            status=form.status.data,
            priority=form.priority.data,

```

```

        progress=form.progress.data or 0,
        estimated_hours=float(form.estimated_hours.data) if form.estimated_hours.data else None,
        start_date=form.start_date.data,
        due_date=form.due_date.data,
        created_by_id=current_user.id,
    )
    task.dependencies = Task.query.filter(Task.id.in_(form.dependencies.data or [])).all()
    db.session.add(task)
    if task.assignee_id and task.assignee_id != current_user.id:
        db.session.flush()
        notify(task.assignee, f"Se te asigno la tarea '{task.title}'",
                link=url_for("tasks.detail", task_id=task.id))
    db.session.commit()
    flash("Tarea creada.", "success")
    return modal_success(url_for("tasks.detail", task_id=task.id))
return modal_form("tasks/_form_modal.html", form, task=None) \
    or render_template("tasks/form.html", form=form, task=None)

@tasks_bp.route("/<int:task_id>/editar", methods=["GET", "POST"])
@login_required
def edit(task_id):
    task = Task.query.get_or_404(task_id)
    if not can_edit_task(current_user, task):
        abort(403)

    form = TaskForm(obj=task)
    _populate_choices(form)
    if form.validate_on_submit():
        prev_assignee = task.assignee_id
        form.populate_obj(task)
        task.assignee_id = form.assignee_id.data or None
        task.estimated_hours = float(form.estimated_hours.data) if form.estimated_hours.data else None
        task.dependencies = Task.query.filter(
            Task.id.in_(form.dependencies.data or []), Task.id != task.id
        ).all()
        if task.status == TaskStatus.DONE and not task.completed_at:
            task.completed_at = datetime.utcnow()
            task.progress = 100
        if task.status != TaskStatus.DONE:
            task.completed_at = None
        if task.assignee_id and task.assignee_id != prev_assignee:
            notify(task.assignee, f"Se te asigno la tarea '{task.title}'",
                    link=url_for("tasks.detail", task_id=task.id))
        db.session.commit()
        flash("Tarea actualizada.", "success")
        return modal_success(url_for("tasks.detail", task_id=task.id))

    if request.method == "GET":
        form.assignee_id.data = task.assignee_id or 0
        form.dependencies.data = [d.id for d in task.dependencies]
    return modal_form("tasks/_form_modal.html", form, task=task) \
        or render_template("tasks/form.html", form=form, task=task)

@tasks_bp.route("/<int:task_id>/estado", methods=["POST"])
@login_required
def change_status(task_id):
    task = Task.query.get_or_404(task_id)
    if not can_edit_task(current_user, task):
        abort(403)
    new_status = request.form.get("status")
    if new_status not in TaskStatus.ALL:
        abort(400)
    task.status = new_status
    if new_status == TaskStatus.DONE:
        task.mark_done()
    else:
        task.completed_at = None
    db.session.commit()
    flash("Estado actualizado.", "success")
    return redirect(request.referrer or url_for("tasks.detail", task_id=task.id))

@tasks_bp.route("/<int:task_id>/eliminar", methods=["POST"])
@login_required
def delete(task_id):
    task = Task.query.get_or_404(task_id)
    if not can_manage_task(current_user, task):
        abort(403)
    project_id = task.project_id
    db.session.delete(task)
    db.session.commit()

```

```
flash("Tarea eliminada.", "info")
return modal_success(url_for("projects.detail", project_id=project_id))
```

app/controllers/teams.py (70 lines)

CRUD de equipos.

```
"""Gestion de equipos (admin y gerentes)."""
from flask import Blueprint, flash, render_template, request, url_for

from ..extensions import db
from ..forms.team_forms import TeamForm
from ..models import Team, User
from ..utils.decorators import manager_required
from ..utils.http import modal_blocked, modal_form, modal_success

teams_bp = Blueprint("teams", __name__)

def _member_choices(form):
    users = User.query.filter(User.active.is_(True)).order_by(User.name).all()
    form.member_ids.choices = [(u.id, f"{u.name} ({u.role.name})") for u in users]

@teams_bp.route("/")
@manager_required
def index():
    teams = Team.query.order_by(Team.name).all()
    return render_template("teams/index.html", teams=teams)

@teams_bp.route("/nuevo", methods=["GET", "POST"])
@manager_required
def create():
    form = TeamForm()
    _member_choices(form)
    if form.validate_on_submit():
        team = Team(name=form.name.data.strip(), description=form.description.data)
        team.members = User.query.filter(User.id.in_(form.member_ids.data or [])).all()
        db.session.add(team)
        db.session.commit()
        flash("Equipo creado.", "success")
        return modal_success(url_for("teams.index"))
    return modal_form("teams/_form_modal.html", form, team=None) \
        or render_template("teams/form.html", form=form, team=None)

@teams_bp.route("/<int:team_id>/editar", methods=["GET", "POST"])
@manager_required
def edit(team_id):
    team = Team.query.get_or_404(team_id)
    form = TeamForm(obj=team)
    _member_choices(form)
    if form.validate_on_submit():
        team.name = form.name.data.strip()
        team.description = form.description.data
        team.members = User.query.filter(User.id.in_(form.member_ids.data or [])).all()
        db.session.commit()
        flash("Equipo actualizado.", "success")
        return modal_success(url_for("teams.index"))
    if request.method == "GET":
        form.member_ids.data = [u.id for u in team.members]
    return modal_form("teams/_form_modal.html", form, team=team) \
        or render_template("teams/form.html", form=form, team=team)

@teams_bp.route("/<int:team_id>/eliminar", methods=["POST"])
@manager_required
def delete(team_id):
    team = Team.query.get_or_404(team_id)
    if team.projects:
        return modal_blocked("El equipo tiene proyectos asociados.",
                             url_for("teams.index"))
    db.session.delete(team)
    db.session.commit()
    flash("Equipo eliminado.", "info")
    return modal_success(url_for("teams.index"))
```

CRUD de usuarios, activar/desactivar y perfil propio.

```

"""Gestion de usuarios (solo administradores)."""
import secrets

from flask import (Blueprint, flash, redirect, render_template, request,
                  url_for)
from flask_login import current_user, login_required

from ..extensions import db
from ..forms.user_forms import ProfileForm, UserForm
from ..models import Role, User
from ..utils.decorators import admin_required
from ..utils.http import modal_blocked, modal_form, modal_success

users_bp = Blueprint("users", __name__)

def _role_choices():
    return [(r.id, r.name.capitalize()) for r in Role.query.order_by(Role.name).all()]

@users_bp.route("/")
@admin_required
def index():
    q = request.args.get("q", "").strip()
    query = User.query
    if q:
        query = query.filter(db.or_(User.name.ilike(f"%{q}%"), User.email.ilike(f"%{q}%")))
    users = query.order_by(User.name).all()
    return render_template("users/index.html", users=users, q=q)

@users_bp.route("/nuevo", methods=["GET", "POST"])
@admin_required
def create():
    form = UserForm()
    form.role_id.choices = _role_choices()
    if form.validate_on_submit():
        email = form.email.data.lower().strip()
        if User.query.filter_by(email=email).first():
            form.email.errors.append("Ese correo ya esta registrado.")
        else:
            user = User(
                name=form.name.data.strip(), email=email,
                job_title=form.job_title.data, role_id=form.role_id.data,
                active=form.is_active.data,
            )
            user.set_password(form.password.data or secrets.token_urlsafe(12))
            db.session.add(user)
            db.session.commit()
            flash("Usuario creado.", "success")
            return modal_success(url_for("users.index"))
    return modal_form("users/_form_modal.html", form, user=None) \
        or render_template("users/form.html", form=form, user=None)

@users_bp.route("/<int:user_id>/editar", methods=["GET", "POST"])
@admin_required
def edit(user_id):
    user = User.query.get_or_404(user_id)
    form = UserForm(obj=user)
    form.role_id.choices = _role_choices()
    if request.method == "GET":
        form.is_active.data = user.is_active

    if form.validate_on_submit():
        email = form.email.data.lower().strip()
        other = User.query.filter(User.email == email, User.id != user.id).first()
        if user.id == current_user.id and not form.is_active.data:
            form.is_active.errors.append("No puedes desactivar tu propia cuenta.")
        elif other:
            form.email.errors.append("Ese correo ya esta en uso.")
        else:
            user.name = form.name.data.strip()
            user.email = email
            user.job_title = form.job_title.data
            user.role_id = form.role_id.data
            user.active = form.is_active.data

```

```

        if form.password.data:
            user.set_password(form.password.data)
            db.session.commit()
            flash("Usuario actualizado.", "success")
            return modal_success(url_for("users.index"))
    return modal_form("users/_form_modal.html", form, user=user) \
        or render_template("users/form.html", form=form, user=user)

@users_bp.route("/<int:user_id>/toggle", methods=["POST"])
@admin_required
def toggle_active(user_id):
    user = User.query.get_or_404(user_id)
    if user.id == current_user.id:
        return modal_blocked("No puedes desactivar tu propia cuenta.",
                             url_for("users.index"))
    user.active = not user.active
    db.session.commit()
    flash(f"Cuenta {'activada' if user.active else 'desactivada'}.", "info")
    return modal_success(url_for("users.index"))

@users_bp.route("/<int:user_id>/eliminar", methods=["POST"])
@admin_required
def delete(user_id):
    user = User.query.get_or_404(user_id)
    if user.id == current_user.id:
        return modal_blocked("No puedes eliminar tu propia cuenta.",
                             url_for("users.index"))
    if user.managed_projects:
        return modal_blocked("Ese usuario gestiona proyectos. Reasignalos antes de eliminarlo.",
                             url_for("users.index"))
    db.session.delete(user)
    db.session.commit()
    flash("Usuario eliminado.", "info")
    return modal_success(url_for("users.index"))

# ----- #
# Perfil propio (cualquier rol)
# ----- #
@users_bp.route("/perfil", methods=["GET", "POST"])
@login_required
def profile():
    form = ProfileForm(obj=current_user)
    if form.validate_on_submit():
        current_user.name = form.name.data.strip()
        current_user.job_title = form.job_title.data
        current_user.avatar_url = form.avatar_url.data
        if form.new_password.data:
            if not current_user.check_password(form.current_password.data or ""):
                flash("La contraseña actual no es correcta.", "danger")
                return render_template("users/profile.html", form=form)
            current_user.set_password(form.new_password.data)
        db.session.commit()
        flash("Perfil actualizado.", "success")
        return redirect(url_for("users.profile"))
    return render_template("users/profile.html", form=form)

```

app/controllers/reports.py (58 lineas)

Pantalla de reportes y exportacion a Excel/PDF.

```

"""Modulo de reportes: graficos + exportacion a Excel / PDF."""
from datetime import datetime

from flask import Blueprint, render_template, request, send_file
from flask_login import current_user

from ..models import Task
from ..models.enums import ProjectStatus
from ..services import export_service, report_service
from ..utils.decorators import manager_required
from ..utils.scope import projects_for, tasks_for

reports_bp = Blueprint("reports", __name__)

def _dataset():
    projects = projects_for(current_user).all()
    status = request.args.get("status", "")

```

```

    if status in ProjectStatus.ALL:
        projects = [p for p in projects if p.status == status]
        project_ids = {p.id for p in projects}
        tasks = [t for t in tasks_for(current_user).all() if t.project_id in project_ids]
        return projects, tasks

@reports_bp.route("/")
@manager_required
def index():
    projects, tasks = _dataset()
    stats = report_service.dashboard_stats(projects, tasks)
    charts = report_service.build_report_bundle(projects, tasks)
    return render_template(
        "reports/index.html",
        stats=stats, charts=charts, projects=projects,
        status=request.args.get("status", ""),
    )

@reports_bp.route("/export/excel")
@manager_required
def export_excel():
    projects, _ = _dataset()
    buf = export_service.projects_to_excel(projects)
    name = f"reporte_proyectos_{datetime.now():%Y%m%d_%H%M}.xlsx"
    return send_file(
        buf, as_attachment=True, download_name=name,
        mimetype="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
    )

@reports_bp.route("/export/pdf")
@manager_required
def export_pdf():
    projects, tasks = _dataset()
    stats = report_service.dashboard_stats(projects, tasks)
    buf = export_service.report_to_pdf(projects, stats)
    name = f"reporte_proyectos_{datetime.now():%Y%m%d_%H%M}.pdf"
    return send_file(buf, as_attachment=True, download_name=name, mimetype="application/pdf")

```

app/controllers/api.py (94 lineas)

Endpoints JSON usados por el Gantt interactivo y las notificaciones.

```

"""Endpoints JSON para el Gantt interactivo y las notificaciones."""
from datetime import date, datetime

from flask import Blueprint, jsonify, request
from flask_login import current_user, login_required

from ..extensions import db
from ..models import Project, Task
from ..models.enums import TaskStatus
from ..services.notification_service import mark_read
from ..utils.permissions import can_edit_task, can_view_project

api_bp = Blueprint("api", __name__)

@api_bp.before_request
@login_required
def _guard():
    """Todas las rutas del API requieren sesion activa."""
    return None

def _parse_date(value):
    if not value:
        return None
    return datetime.fromisoformat(value.replace("Z", "")[:19]).date()

@api_bp.get("/projects/<int:project_id>/ganttt")
def project_ganttt(project_id):
    project = Project.query.get_or_404(project_id)
    if not can_view_project(current_user, project):
        return jsonify(error="forbidden"), 403
    return jsonify(tasks=[t.to_ganttt() for t in project.tasks])
@api_bp.post("/tasks/<int:task_id>/schedule")
def reschedule_task(task_id):

```

```

        """Persiste el arrastrar/soltar de fechas desde Frappe Gantt."""
        task = Task.query.get_or_404(task_id)
        if not can_edit_task(current_user, task):
            return jsonify(error="forbidden"), 403
        data = request.get_json(silent=True) or {}
        try:
            start = _parse_date(data.get("start"))
            end = _parse_date(data.get("end"))
        except ValueError:
            return jsonify(error="invalid_date"), 400
        if start:
            task.start_date = start
        if end:
            task.due_date = end
        db.session.commit()
        return jsonify(ok=True, task=task.to_gantt())

@api_bp.post("/tasks/<int:task_id>/progress")
def update_progress(task_id):
    task = Task.query.get_or_404(task_id)
    if not can_edit_task(current_user, task):
        return jsonify(error="forbidden"), 403
    data = request.get_json(silent=True) or {}
    try:
        progress = max(0, min(100, int(data.get("progress", task.progress))))
    except (TypeError, ValueError):
        return jsonify(error="invalid"), 400
    task.progress = progress
    if progress == 100 and task.status != TaskStatus.DONE:
        task.mark_done()
    db.session.commit()
    return jsonify(ok=True, progress=task.progress, status=task.status)

@api_bp.get("/notifications")
def notifications():
    items = current_user.notifications[:20]
    return jsonify(
        unread=sum(1 for n in current_user.notifications if not n.is_read),
        items=[
            {
                "id": n.id, "message": n.message, "link": n.link,
                "category": n.category, "is_read": n.is_read,
                "created_at": n.created_at.isoformat(),
            }
            for n in items
        ],
    )

@api_bp.post("/notifications/read")
def read_notifications():
    data = request.get_json(silent=True) or {}
    count = mark_read(current_user, data.get("id"))
    return jsonify(ok=True, marked=count)

```

6.7 JavaScript (logica de cliente)

app/static/js/app.js (79 lineas)

Utilidades globales: token CSRF, tema claro/oscuro, sidebar movil, confirmaciones y el badge de notificaciones.

```
// Utilidades globales: tema, sidebar, notificaciones, confirmaciones.
(function () {
  "use strict";

  window.GP = {
    csrf: function () {
      var m = document.querySelector('meta[name="csrf-token"]');
      return m ? m.getAttribute("content") : "";
    },
    postJSON: function (url, body) {
      return fetch(url, {
        method: "POST",
        headers: { "Content-Type": "application/json", "X-CSRFToken": window.GP.csrf() },
        body: JSON.stringify(body || {}),
      }).then(function (r) { return r.json(); });
    },
    chartColors: function () {
      var dark = document.documentElement.getAttribute("data-bs-theme") === "dark";
      return {
        grid: dark ? "rgba(255,255,255,.08)" : "rgba(0,0,0,.06)",
        text: dark ? "#8b949e" : "#6b7280",
      };
    },
  };
};

// ----- Tema claro / oscuro -----
var root = document.documentElement;
var saved = null;
try { saved = localStorage.getItem("gp-theme"); } catch (e) {}
if (saved) root.setAttribute("data-bs-theme", saved);

var themeBtn = document.getElementById("themeToggle");
if (themeBtn) {
  themeBtn.addEventListener("click", function () {
    var next = root.getAttribute("data-bs-theme") === "dark" ? "light" : "dark";
    root.setAttribute("data-bs-theme", next);
    try { localStorage.setItem("gp-theme", next); } catch (e) {}
    document.dispatchEvent(new CustomEvent("gp:themechange", { detail: next }));
  });
}

// ----- Sidebar movil -----
var sidebar = document.getElementById("sidebar");
var backdrop = document.getElementById("sidebarBackdrop");
function openSidebar(open) {
  if (!sidebar) return;
  sidebar.classList.toggle("open", open);
  if (backdrop) backdrop.classList.toggle("show", open);
}
["sidebarToggle"].forEach(function (id) {
  var el = document.getElementById(id);
  if (el) el.addEventListener("click", function () { openSidebar(true); });
});
["sidebarClose", "sidebarBackdrop"].forEach(function (id) {
  var el = document.getElementById(id);
  if (el) el.addEventListener("click", function () { openSidebar(false); });
});

// ----- Confirmacion en formularios peligrosos -----
document.querySelectorAll("form.js-confirm").forEach(function (f) {
  f.addEventListener("submit", function (e) {
    if (!window.confirm(f.dataset.message || "Confirmas esta accion?")) e.preventDefault();
  });
});

// ----- Notificaciones: marcar como leidas -----
var markAll = document.getElementById("markAllRead");
if (markAll) {
  markAll.addEventListener("click", function () {
    window.GP.postJSON("/api/notifications/read", {}).then(function () {
      document.querySelectorAll(".notif-item.unread").forEach(function (n) {
        n.classList.remove("unread");
      });
    });
    var dot = document.getElementById("notifDot");
    if (dot) dot.remove();
  });
}
```

```

    });
  });
}
})();

```

app/static/js/modal-form.js (170 lineas)

El motor del patron de modales AJAX: abre el formulario por fetch(), lo envia, muestra errores de validacion o navega al exito, y arma los modales de confirmacion.

```

// Formularios y confirmaciones en modal (sin cambiar de pagina).
// - [data-modal-form][data-url] -> carga un formulario por AJAX y lo envia
// - [data-modal-confirm][data-url][data-message] -> confirma y hace POST
(function () {
  "use strict";

  var modalEl = document.getElementById("ajaxModal");
  if (!modalEl || typeof bootstrap === "undefined") return;

  var bsModal = new bootstrap.Modal(modalEl);
  var dialog = modalEl.querySelector(".modal-dialog");

  function csrf() {
    return (window.GP && window.GP.csrf && window.GP.csrf()) || "";
  }

  function toast(message, type) {
    var stack = document.getElementById("toastStack");
    if (!stack) { window.alert(message); return; }
    var el = document.createElement("div");
    el.className = "alert alert-" + (type || "danger") + " alert-dismissible shadow-sm gp-toast";
    el.setAttribute("role", "alert");
    el.innerHTML = message +
      '<button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Cerrar"></button>';
    stack.appendChild(el);
    setTimeout(function () {
      el.classList.add("out");
      setTimeout(function () { el.remove(); }, 300);
    }, 4500);
  }
  if (window.GP) window.GP.toast = toast;

  var spinnerContent =
    '<div class="modal-content gp-modal">' +
    '<div class="modal-body text-center py-5">' +
    '<span class="spinner-border text-primary" role="status"></span></div></div>';

  function busy(btn, on) {
    if (!btn) return;
    if (on) {
      btn.dataset.html = btn.innerHTML;
      btn.disabled = true;
      btn.innerHTML = '<span class="spinner-border spinner-border-sm"></span>';
    } else {
      btn.disabled = false;
      if (btn.dataset.html) btn.innerHTML = btn.dataset.html;
    }
  }

  function bindForm(form) {
    if (!form) return;
    form.addEventListener("submit", function (e) {
      e.preventDefault();
      var submitBtn = form.querySelector('[type="submit"]');
      busy(submitBtn, true);

      fetch(form.action, {
        method: "POST",
        headers: { "X-Requested-With": "XMLHttpRequest", "X-CSRFToken": csrf() },
        body: new FormData(form),
      })
        .then(function (res) {
          var ct = res.headers.get("content-type") || "";
          if (res.ok && ct.indexOf("application/json") !== -1) {
            return res.json().then(function (data) {
              bsModal.hide();
              window.location = data.redirect || window.location.href;
            });
          }
        })
        // 422: el servidor devuelve el formulario con los errores marcados
        return res.text().then(function (html) {

```

```

        dialog.innerHTML = html;
        bindForm(dialog.querySelector("form"));
    });
}
).catch(function () {
    busy(submitBtn, false);
    toast("No se pudo guardar. Revisa tu conexion e intenta de nuevo.");
});
});
}

function openForm(url) {
    dialog.innerHTML = spinnerContent;
    bsModal.show();
    fetch(url, { headers: { "X-Requested-With": "XMLHttpRequest" } })
        .then(function (res) {
            if (!res.ok) throw new Error(res.status);
            return res.text();
        })
        .then(function (html) {
            dialog.innerHTML = html;
            bindForm(dialog.querySelector("form"));
            var first = dialog.querySelector("input:not([type=hidden]), textarea, select");
            if (first) first.focus();
        })
        .catch(function () {
            bsModal.hide();
            toast("No se pudo abrir el formulario.");
        });
}

function openConfirm(opts) {
    var warn = opts.variant === "warn";
    var headClass = warn ? "gp-modal" : "gp-modal gp-modal-danger";
    var btnClass = warn ? "btn btn-primary" : "btn btn-danger";
    var headIcon = warn ? "fa-circle-question" : "fa-triangle-exclamation";
    var goIcon = opts.icon || (warn ? "fa-check" : "fa-trash-can");
    dialog.innerHTML =
        '<div class="modal-content ' + headClass + '">' +
        '<div class="modal-header"><h5 class="modal-title">' +
        '<i class="fa-solid ' + headIcon + ' me-2"></i><span data-slot="title"></span>' +
        '</h5><button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal"></button></div>' +
        '<div class="modal-body" data-slot="message"></div>' +
        '<div class="modal-footer">' +
        '<button type="button" class="btn btn-light" data-bs-dismiss="modal">Cancelar</button>' +
        '<button type="button" class="' + btnClass + ' ' + goIcon + '" data-confirm-go>' +
        '<i class="fa-solid ' + goIcon + ' me-1"></i><span data-slot="action"></span></button></div></div>';
    dialog.querySelector('[data-slot="title"]').textContent = opts.title || "Confirmar";
    dialog.querySelector('[data-slot="message"]').textContent =
        opts.message || "¿Confirmas esta accion?";
    dialog.querySelector('[data-slot="action"]').textContent = opts.action || "Eliminar";
    bsModal.show();

    dialog.querySelector("[data-confirm-go]").addEventListener("click", function () {
        var go = this;
        busy(go, true);
        fetch(opts.url, {
            method: "POST",
            headers: { "X-Requested-With": "XMLHttpRequest", "X-CSRFToken": csrf() },
        })
            .then(function (res) {
                return res.json().catch(function () { return { ok: res.ok }; });
            })
            .then(function (data) {
                bsModal.hide();
                if (data.ok === false) {
                    toast(data.error || "No se pudo completar la accion.", "warning");
                } else {
                    window.location = data.redirect || window.location.href;
                }
            })
            .catch(function () {
                busy(go, false);
                toast("No se pudo completar la accion.");
            });
    });
}

document.addEventListener("click", function (e) {
    var opener = e.target.closest("[data-modal-form]");
    if (opener) {
        e.preventDefault();
        openForm(opener.getAttribute("data-url") || opener.getAttribute("href"));
    }
});

```

```

        return;
    }
    var confirmer = e.target.closest("[data-modal-confirm]");
    if (confirmer) {
        e.preventDefault();
        openConfirm({
            url: confirmer.getAttribute("data-url"),
            title: confirmer.getAttribute("data-title"),
            message: confirmer.getAttribute("data-message"),
            action: confirmer.getAttribute("data-action"),
            variant: confirmer.getAttribute("data-variant"),
            icon: confirmer.getAttribute("data-icon"),
        });
    }
});
})();

```

app/static/js/gantt.js (41 lineas)

Inicializa Frappe Gantt y envia el arrastrar-y-soltar de fechas a la API.

```

(function () {
    "use strict";
    var cfg = window.GP_GANTT;
    if (!cfg || !cfg.tasks.length || typeof Gantt === "undefined") return;

    var gantt = new Gantt("#gantt", cfg.tasks, {
        view_mode: "Week",
        date_format: "YYYY-MM-DD",
        language: "es",
        readonly: !cfg.canEdit,
        popup_trigger: "click",
        custom_popup_html: function (task) {
            return (
                '<div class="gantt-popup p-2">' +
                '<strong>' + task.name + '</strong><br>' +
                '<small>' + task._start.toISOString().slice(0, 10) +
                ' &rarr; ' + task._end.toISOString().slice(0, 10) + '</small><br>' +
                '<small>Avance: ' + task.progress + '%</small>' +
                '</div>'
            );
        },
        on_date_change: function (task, start, end) {
            if (!cfg.canEdit) return;
            var url = cfg.scheduleUrl.replace(/0$/ , task.id);
            window.GP.postJSON(url, {
                start: start.toISOString().slice(0, 10),
                end: end.toISOString().slice(0, 10),
            }).then(function (res) {
                if (!res.ok) alert("No se pudo guardar la nueva fecha.");
            });
        },
    });
})();

document.querySelectorAll("#ganttModes button").forEach(function (btn) {
    btn.addEventListener("click", function () {
        document.querySelectorAll("#ganttModes button").forEach(function (b) { b.classList.remove("active"); });
        btn.classList.add("active");
        gantt.change_view_mode(btn.dataset.mode);
    });
});
})();

```

app/static/js/dashboard.js (24 lineas)

Dibuja los dos graficos del dashboard con Chart.js.

```
(function () {
  "use strict";
  var data = window.GP_CHARTS;
  if (!data || typeof Chart === "undefined") return;
  var c = window.GP.chartColors();
  Chart.defaults.color = c.text;
  Chart.defaults.font.family = "'Inter', sans-serif";

  new Chart(document.getElementById("chartMonthly"), {
    type: "line",
    data: data.monthly,
    options: {
      responsive: true,
      plugins: { legend: { display: false } },
      scales: { y: { beginAtZero: true, grid: { color: c.grid } }, x: { grid: { display: false } } },
    },
  });

  new Chart(document.getElementById("chartStatus"), {
    type: "doughnut",
    data: data.status,
    options: { responsive: true, plugins: { legend: { position: "bottom" } }, cutout: "62%" },
  });
})();
```

app/static/js/reports.js (21 lineas)

Dibuja los graficos de la pantalla de reportes.

```
(function () {
  "use strict";
  var d = window.GP_REPORTS;
  if (!d || typeof Chart === "undefined") return;
  var c = window.GP.chartColors();
  Chart.defaults.color = c.text;
  Chart.defaults.font.family = "'Inter', sans-serif";

  var bar = function (id, cfg) {
    var el = document.getElementById(id);
    if (el) new Chart(el, cfg);
  };

  var gridScales = { y: { beginAtZero: true, grid: { color: c.grid } }, x: { grid: { display: false } } };

  bar("cProgress", { type: "bar", data: d.progress, options: { indexAxis: "y", plugins: { legend: { display: false } }, scales: gridScales });
  bar("cWorkload", { type: "bar", data: d.workload, options: { plugins: { legend: { display: false } }, scales: gridScales });
  bar("cStatus", { type: "doughnut", data: d.status, options: { plugins: { legend: { position: "bottom" } }, cutout: "62%" });
  bar("cCompliance", { type: "pie", data: d.compliance, options: { plugins: { legend: { position: "bottom" } } });
  bar("cPriority", { type: "doughnut", data: d.priority, options: { plugins: { legend: { position: "bottom" } }, cutout: "62%" });
  bar("cMonthly", { type: "line", data: d.monthly, options: { plugins: { legend: { display: false } }, scales: gridScales });
})();
```

7 · Flujos clave, paso a paso

7.1 Autenticacion y alcance por rol

- El primer usuario que se registra se convierte en **admin** automaticamente.
- La contraseña se guarda con pbkdf2:sha256; nunca en texto plano (ver seccion 2).
- Cada formulario lleva un token CSRF (Flask-WTF) excepto el blueprint de API, que viaja autenticado por sesion y cabecera propia.
- En cada ruta se combinan dos capas de permiso: **utils/scope.py** filtra que filas puede ver el usuario en la consulta, y **utils/permissions.py** decide si puede editar/borrar un registro concreto.

7.2 CRUD en modal (el patron mas reutilizado del proyecto)

Crear, editar y eliminar en proyectos, tareas, equipos y usuarios funciona igual en las cuatro secciones, sin recargar la pagina. El truco: **la misma ruta sirve tres formatos distintos** segun quien pregunte:

Paso	Que pasa
1. Clic en "Nuevo" / "Editar"	JS (modal-form.js) hace un GET a la misma URL, con la cabecera X-Requested-With: XMLHttpRequest.
2. El controlador detecta AJAX	wants_json() ve la cabecera y devuelve solo el fragmento _form_modal.html en vez de la pagina completa.
3. Se envia el formulario	El JS intercepta el submit y hace POST por fetch() a la misma accion de siempre.
4. Validacion fallida	modal_form() vuelve a renderizar el fragmento con los errores de WTForms ya marcados (HTTP 422); el JS reemplaza el contenido del modal.
5. Exito	modal_success() responde JSON {ok: true, redirect: ...}; el JS cierra el modal y navega, donde el flash de siempre confirma la accion.
6. Accion bloqueada	modal_blocked() responde {ok: false, error: "..."} y el JS lo muestra como un toast, sin cerrar la pagina.
Sin JavaScript	La URL es la misma: si JS falla, el enlace navega normal y la vista devuelve la pagina completa de siempre. Mejora progresiva, no dependencia.

7.3 Cronograma de Gantt: interactivo y exportable

- En pantalla se dibuja con **Frappe Gantt**, alimentada por `Task.to_gantt()`.
- Arrastrar una barra dispara un POST a `/api/tasks/<id>/schedule`, validado con `can_edit_task`, que guarda las nuevas fechas.
- El boton **PDF** vuelve a calcular la escala de tiempo y dibuja cada barra con `reportlab.graphics`, coloreada por estado y con el % de avance superpuesto.

7.4 Reportes, exportacion y notificaciones

- `reports.index` arma varios datasets con `report_service`; el HTML los pinta con `Chart.js`.
- Los mismos datos se exportan a Excel (`openpyxl`, dos hojas) y PDF (tablas con `ReportLab`).
- `flask scan-notifications` recorre tareas por vencer/vencidas y notifica al responsable, con una `dedup_key` para no repetir el mismo aviso -pensado para un cron cada 30 min.

8 · Para explicarlo: preguntas frecuentes

P: Por que Flask y no Django u otro framework mayor

R: Flask deja elegir cada pieza (ORM, formularios, migraciones) y el proyecto necesitaba demostrar exactamente esas decisiones: como se arma la capa de permisos, el patron de modales, etc. Con Django gran parte vendria resuelto "de fabrica".

P: Como evitas que un colaborador vea proyectos de otra persona

R: En dos capas: la consulta ya sale filtrada por rol (utils/scope.py), y ademas cada vista de detalle vuelve a comprobar el permiso puntual (utils/permissions.py) antes de mostrar o modificar el recurso. Aunque alguien adivine la URL, la segunda capa responde 403.

P: Donde estan las contraseñas y como se protegen

R: Nunca se guardan en texto plano: models/user.py usa generate_password_hash (pbkdf2:sha256) al crear la cuenta y check_password_hash al iniciar sesion. Las credenciales de prueba de la seccion 2 solo existen si corriste flask seed.

P: Como persiste el arrastrar-y-soltar del Gantt

R: Frappe Gantt dispara un evento en el cliente; gantt.js hace POST a /api/tasks/<id>/schedule con las nuevas fechas en JSON. La ruta valida permisos, actualiza el modelo y devuelve el estado actualizado.

P: Por que modales AJAX en vez de una pagina por formulario

R: Para que crear o editar no interrumpa el contexto. El patron reutiliza los mismos formularios de WTForms y el mismo controlador: solo cambia el formato de respuesta segun una cabecera, sin codigo duplicado entre la version con y sin JavaScript.

P: Que pasaria si el proyecto creciera: que cambiarias primero

R: Sacar las migraciones de "crear tablas a mano" (flask init-db) a Flask-Migrate/Alembic versionado, mover el escaneo de notificaciones a una cola en vez de un cron simple, y paginar/cachear los reportes cuando haya muchos proyectos.